

DUOC UC - Escuela de informática y telecomunicaciones

Propuesta de Proyecto Especificación de Requisitos de Software

Proyecto: [Insertar Nombre de Proyecto]

Revisión: **[01]**

Contenido

<i>DUOC UC - Escuela de informática y telecomunicaciones</i>	
1 FICHA DEL DOCUMENTO	3
1. INTRODUCCIÓN	3
1.1. PROPÓSITO	3
1.2. ÁMBITO DEL SISTEMA.....	4
1.3. DEFINICIONES, ACRÓNIMOS Y ABREVIATURAS	4
1.4. REFERENCIAS	4
1.5. VISIÓN GENERAL DEL DOCUMENTO	4
2. DESCRIPCIÓN GENERAL	5
2.1. PERSPECTIVA DEL PRODUCTO.....	5
2.2. FUNCIONES DEL PRODUCTO	5
2.3. CARACTERÍSTICAS DE LOS USUARIOS	5
2.4. RESTRICCIONES	6
2.5. SUPOSICIONES Y DEPENDENCIAS.....	6
2.6. REQUISITOS FUTUROS	6
3. REQUISITOS ESPECÍFICOS	7
3.1. REQUISITOS COMUNES DE LAS INTERFACES	8
3.1.1 <i>Interfaces de usuario</i>	8
3.2. REQUISITOS FUNCIONALES.....	8
3.4 OTROS REQUISITOS.....	9
4. PROPUESTA DE PLANIFICACIÓN	9
4.1. DESCRIPCIÓN GENERAL ACERCA DE LA PLANIFICACIÓN	9

4.1.2	Definición del Equipo de Trabajo.....	9
4.1.3	Definición de Actividades principales del Proyecto	9
4.1.4	tablero Kanban o carta gantt	10
4.1.5	Carta Gantt	10
5.	ANEXOS	10
	5.2 Matriz Especificación de Requerimientos	10

Ficha del documento

Fecha	Revisión	Autor	Modificación
[Fecha]	[Rev]	[Descripción]	[Descripción]
[Fecha]	[Rev]	[Descripción]	[Descripción]

Documento validado por las partes en fecha: [Fecha]

Integrantes:

Nombre Integrante del Equipo	Rol Definido
Alejandro Naranjo	Analista de Sistemas
Christian Montecinos	Diseñador de Software
Benjamin Vazques	Jefe de proyecto

1. Introducción

En Este Informe se detallo la Especificación de Requisitos de Software (ERS) para el nuevo **Sistema de Gestión Bibliotecaria de la Universidad UNILIB**. El propósito es proporcionar una descripción completa de las funciones y restricciones del sistema para modernizar los procesos actuales, que se encuentran desactualizados y son manuales. Está dirigido al equipo de desarrollo, administración de la biblioteca y stakeholders universitarios.

1.1. Propósito

Definir los requisitos para el desarrollo del **Sistema de Gestión Bibliotecaria UNILIB**. Su objetivo es formalizar las necesidades del cliente y servir de guía técnica para los desarrolladores y el área de QA, asegurando que el software resuelva la ineficiencia de los procesos manuales actuales bajo el estándar **IEEE 830**.

1.2. Ámbito del Sistema

Sistema de Gestión Bibliotecaria UNILIB

Automatizará el préstamo y devolución de libros, gestionará el catálogo en línea, administrará perfiles de usuario y generará reportes estadísticos para la toma de decisiones.

No gestionará la compra de activos físicos ni el pago de nómina del personal.

Reducir las filas en atención, evitar el extravío de material bibliográfico y permitir el acceso remoto al catálogo desde dispositivos personales.

1.3. Definiciones, Acrónimos y Abreviaturas

ERS: Especificación de Requisitos de Software.

IEEE 830: Estándar para la especificación de requisitos de software.

RBAC: Control de acceso basado en roles.

1.4. Referencias

1.5. Visión General del Documento

El documento se divide en: Descripción General (contexto del negocio), Requisitos Específicos (lista de RF y RNF) y Propuesta de Planificación (tiempos, equipo y costos).

2. Descripción General

El sistema **UNILIB** es un producto de software nuevo e independiente, diseñado para sustituir los procesos manuales de registro en papel y planillas aisladas. Aunque es una solución autónoma, se proyecta su integración futura con la base de datos central de alumnos de la universidad para

validar identidades y estados académicos en tiempo real. El sistema operará en un entorno web, permitiendo el acceso tanto desde la red interna de la biblioteca como de forma remota para los estudiantes.

2.1. Perspectiva del Producto

El sistema UNILIB será una solución integral e independiente que reemplazará los procesos manuales actuales. Se diseñará con una arquitectura capaz de integrarse en el futuro con el sistema central de registros de la universidad.

2.2. Funciones del Producto

Gestión de Usuarios: Administración de perfiles y roles.

Gestión de Inventario: Registro y ubicación de materiales.

Control de Circulación: Préstamos y devoluciones con notificaciones.

Consulta Externa: Catálogo en línea con filtros de búsqueda.

Inteligencia de Negocio: Reportes estadísticos de uso.

2.3. Características de los Usuarios

Usuarios	Perfil – o tipo	descripción
Estudiante / Docente	Externo	Consulta catálogo, reserva libros y recibe notificaciones.
Bibliotecario	Operativo	Gestiona préstamos, devoluciones y el inventario físico.
Administrador	Gestion	Administra roles, usuarios y genera reportes estadísticos.

2.4. Restricciones

El desarrollo debe finalizar en un plazo de 4 meses.

- Debe ser compatible con los navegadores Chrome, Firefox y Safari.
- Cumplimiento estricto con la norma IEEE 830 para la documentación.

2.5. Suposiciones y Dependencias

- **Suposiciones:** Se asume que todos los usuarios finales (estudiantes y funcionarios) tienen conocimientos básicos de navegación web. Se asume que la infraestructura de servidores de la Universidad UNILIB cuenta con la capacidad para alojar una base de datos relacional y un servidor web activo 24/7.
- **Dependencias:** El sistema depende directamente de la base de datos central de la Universidad para la validación de la vigencia de los alumnos y docentes. También depende de la conectividad a internet de la institución para que el catálogo sea consultable externamente.

2.6. Requisitos Futuros

Integración con bases de datos internacionales: Conexión con bibliotecas digitales externas (como IEEE o Springer) para acceso a e-books.

- **Módulo de pago en línea:** Implementación de una pasarela de pagos (Webpay) para la cancelación de multas por atraso.
- **App Móvil Nativa:** Desarrollo de aplicaciones en iOS y Android con notificaciones push y escaneo de códigos QR para préstamos rápidos.
- **Chatbot de asistencia:** Un asistente virtual basado en IA para resolver dudas frecuentes sobre disponibilidad y horarios.

3.1.1 Interfaces de usuario

- **Interfaces de Usuario:** El sistema será una aplicación Web con diseño **Responsive**. La navegación será a través de un menú lateral para funcionarios y una barra de búsqueda central tipo Google para alumnos. Se utilizarán mensajes modales para confirmar acciones (ej: 'Libro prestado con éxito').
- **Interfaces de Hardware:** El sistema debe ser compatible con lectores de códigos de barra USB/Bluetooth para la lectura de folios de libros y credenciales. Se requiere un servidor con al menos 8GB de RAM y procesador Quad-Core para el hosting.
- **Interfaces de Software:** El sistema correrá sobre un stack LIMP (Linux, Nginx, MySQL, PHP/Python). Se comunicará con el servicio de correo de la Universidad mediante el protocolo SMTP para el envío de alertas.

3.1.2 Interfaces de hardware

- **Lectores de códigos de barra:** El sistema debe ser compatible con lectores láser vía USB para la identificación rápida de libros y carnets de biblioteca.

- **Servidores:** Procesador de al menos 4 núcleos y 8GB de RAM para garantizar la fluidez.

3.1.3 Interfaces de software

- **Sistema Operativo:** Compatible con entornos Linux (Ubuntu Server) o Windows Server.
- **Base de Datos:** PostgreSQL 14 o superior para la gestión de datos relacionales.
- **Navegadores:** Optimizado para versiones recientes de Google Chrome, Mozilla Firefox y Microsoft Edge.

3.1.4 Interfaces de comunicación

- **Protocolo:** Uso de **HTTPS** con certificados SSL para la transferencia segura de datos de los usuarios.
- **Correo:** Integración con el servidor SMTP de la universidad para el envío masivo de recordatorios.

3.2 Requisitos funcionales (Los 15 solicitados)

1. **RF1:** Registro y gestión de perfiles de usuario (CRUD).
2. **RF2:** Registro y catalogación de material bibliográfico (CRUD).
3. **RF3:** Motor de búsqueda con filtros por autor, título, ISBN y materia.
4. **RF4:** Control de stock físico (ejemplares disponibles vs. totales).
5. **RF5:** Registro de préstamos vinculando ID de libro y RUT de usuario.
6. **RF6:** Cálculo automático de fecha de entrega según perfil (ej: alumno 3 días, docente 7 días).
7. **RF7:** Registro de devoluciones y actualización inmediata de disponibilidad.
8. **RF8:** Bloqueo automático de préstamos a usuarios con multas o libros atrasados.
9. **RF9:** Gestión de reservas de libros que están en préstamo.
10. **RF10:** Generación de avisos preventivos vía correo 24 horas antes del vencimiento.
11. **RF11:** Reporte mensual de libros más solicitados (formato PDF/Excel).
12. **RF12:** Gestión de baja de libros (extravío, daño o obsolescencia).
13. **RF13:** Administración de niveles de acceso (Roles: Admin, Bibliotecario, Alumno).
14. **RF14:** Panel de historial de préstamos para que el alumno revise sus movimientos.

15.**RF15:** El sistema permitirá parametrizar el valor de la multa diaria por retraso

3.3 Requisitos No Funcionales (Los 5 solicitados)

1. **RNF1 (Seguridad):** Autenticación obligatoria y encriptación de datos sensibles (RUT y claves).
2. **RNF2 (Eficiencia):** Las consultas al catálogo deben mostrar resultados en un tiempo máximo de 3 segundos
3. **RNF3 (Usabilidad):** La interfaz debe ser responsiva (adaptable a celulares y tablets).
4. **RNF4 (Disponibilidad):** El sistema debe garantizar una disponibilidad del 99.9% durante el horario de operación de la biblioteca
5. **RNF5 (Estándar):** El desarrollo debe seguir las normas de documentación IEEE 830 exigidas.

3.4 Otros Requisitos

- **Legal:** Cumplimiento con la Ley de Protección de Datos Personales en el manejo de la información de los estudiantes.
- **Copia de Seguridad:** Respaldo automatizado de la base de datos cada 24 horas a las 03:00 AM.

4. Propuesta de Planificación

4.1 Descripción general acerca de la Planificación

La planificación se basa en los estándares del **PMBOK (PMI)**, centrándose en la Gestión del Cronograma y los Recursos. Se utilizará una metodología **Híbrida**, aplicando rigor en la captura de requisitos (IEEE 830) y flexibilidad en el desarrollo mediante Sprints de 2 semanas, asegurando entregas de valor constantes al personal de la biblioteca.

4.1.2 Definición del Equipo de Trabajo

Rol	Funciones
Project Manager	Gestión de plazos, costos y comunicación con UNILIB.

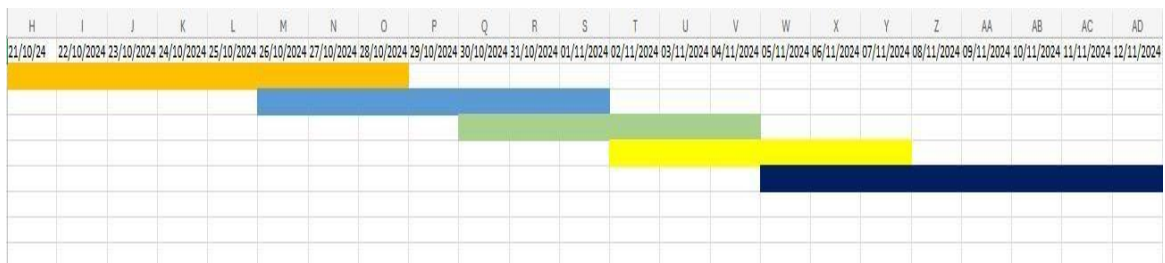
Analista de Sistemas	Definición de requerimientos y modelado de datos.
Desarrollador (QA)	Programación de módulos y ejecución de pruebas de calidad.

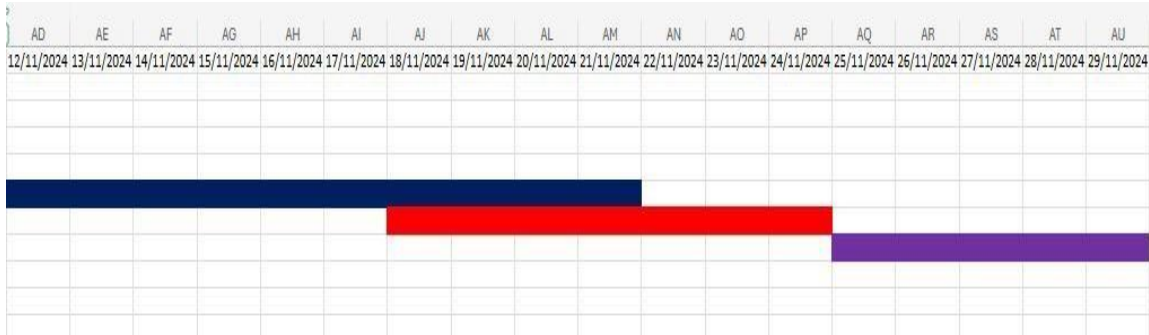
4.1.3 Definición de Actividades principales del Proyecto Se basan en los estándares del **PMI** y la **Ingeniería de Software**:

1. **Levantamiento y Análisis:** Validación de requerimientos con el cliente.
2. **Arquitectura y Diseño:** Creación de diagramas UML (Clases, Casos de Uso).
3. **Construcción (Sprint 1 y 2):** Desarrollo del núcleo del sistema y base de datos.
4. **Control de Calidad (QA):** Pruebas unitarias e integrales.
5. **Despliegue y Cierre:** Capacitación a bibliotecarios y entrega de documentación técnica.

4.1.4 tablero Kanban o carta gantt

B	C	D	E	F	G
Actividad	Responsable	Fecha Inicio	Duración (días)	Fecha fin	Costo Estimado
Levantamiento y Análisis de Requisitos	Analista	21/10/2024	7	28/10/2024	500000
Modelado UML	Analista	26/10/2024	6	01/11/2024	600000
Diseño de Arquitectura	Arquitecto	30/10/2024	5	04/11/2024	550000
Configuración de Base de Datos	Desarrollador	02/11/2024	5	07/11/2024	450000
Desarrollo de Prototipo Funcional	Desarrollador	05/11/2024	15	20/11/2024	1200000
Pruebas Unitarias y de Seguridad	QA Tester	18/11/2024	6	24/11/2024	450000
Capacitación y Entrega Final	Jefe Proy	25/11/2024	4	29/11/2024	450000





4.1.5 Carta Gantt

Se ha optado por una visualización de calendario extendido para garantizar la trazabilidad de los hitos de entrega. Cada barra representa la duración exacta de las fases del ciclo de vida, alineadas con el presupuesto detallado en la columna de Costo Estimado.

e

1 Diseño y Representación Arquitectura 4+1

La arquitectura de nuestro proyecto Sistema de Gestión Bibliotecaria UNILIB está representada siguiendo el enfoque del framework 4+1 y las recomendaciones del proceso unificado. Como equipo de desarrollo, elegimos este enfoque porque nos permite fragmentar la complejidad del sistema en perspectivas independientes, asegurando que tanto los requisitos de los usuarios

(como el préstamo de libros) como las restricciones tecnológicas (el uso de servidores cloud) queden correctamente respaldados. Las vistas incluidas en esta versión del documento son:

Vista de Casos de Uso y Escenarios de Calidad: Describe el comportamiento del sistema desde el punto de vista de nuestros usuarios (Estudiantes y Bibliotecarios). Muestra las funciones prioritarias y cómo los atributos de calidad (como la velocidad de búsqueda) guían nuestro diseño.

Vista Lógica: Detalla la organización interna de nuestro código fuente. Estructuramos el software en capas lógicas para asegurar que la base de datos esté completamente aislada de las pantallas de la aplicación.

Vista de Procesos: Enfocada en el rendimiento en tiempo de ejecución. Muestra cómo nuestro sistema gestiona la concurrencia cuando muchos alumnos intentan buscar libros al mismo tiempo en épocas de exámenes.

Vista de Implementación / Desarrollo: Muestra cómo organizamos nuestros archivos de código, librerías y submódulos de software en nuestro entorno de desarrollo para poder trabajar en paralelo como equipo de programadores.

Vista de Despliegue: Describe la distribución física de la infraestructura (servidores de aplicación y clúster de base de datos) donde operará el sistema de la universidad.

Describe restricciones tecnológicas, normativas, estándares, etc., los cuales influyen sobre las decisiones arquitectónicas, del producto y del proceso de desarrollo.

1.1 Vista Escenarios , de Casos de Uso

Riesgo Transaccional: Procesos donde una falla puede corromper el stock real de libros o generar duplicidad en el inventario.

Densidad de Reglas de Negocio: Flujos que requieren consultar múltiples entidades (Alumnos, Multas, Estados de Matrícula).

Distribución Física: Operaciones que obligan a comunicar múltiples nodos de la vista de despliegue en tiempo real.

1.1.1 Modelo de Casos de Uso

El modelo de casos de uso de UNILIB se elaboró aplicando el principio de minimalidad del Proceso Unificado: cada actor aparece sólo si interactúa directamente con la frontera del sistema y cada caso de uso responde a un objetivo medible del negocio. Esto evita inflar el diagrama con interacciones internas que pertenecen a la vista lógica.

Se identificaron cuatro actores principales —Estudiante, Docente, Bibliotecario y Administrador— y un actor externo (Servicios Académicos) que valida la vigencia de matrículas. Los casos de uso fueron agrupados en cuatro familias funcionales: acceso (autenticación), circulación (préstamos y devoluciones), consulta (catálogo y reservas) y administración (CRUD de libros, usuarios y reportes).

El modelo está estructurado para centralizar los flujos de control de inventario y atención de usuarios en la biblioteca UNILIB.

1.1.2 Diagrama de Casos de Uso general

El siguiente diagrama de contexto delimita la frontera del sistema y muestra las interacciones de alto nivel entre los actores y la plataforma. Permite a los stakeholders entender el alcance de UNILIB sin necesidad de leer cada caso de uso individual.

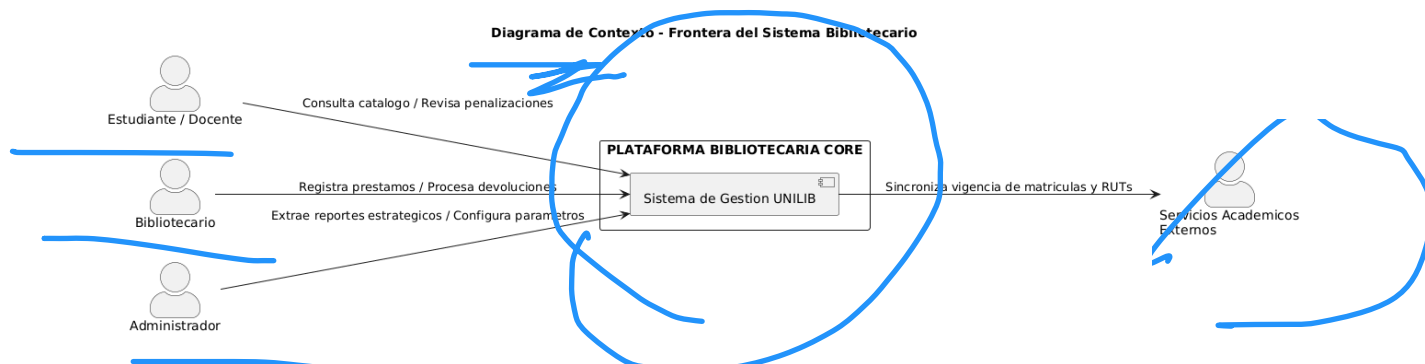


Ilustración 1. Diagrama de Contexto – Frontera del Sistema UNILIB

A continuación se presenta el modelo de casos de uso completo. Las dependencias<<include>>marcan los pasos obligatorios (por ejemplo, Iniciar sesión es prerequisite de Solicitar préstamo y Registrar devolución), garantizando coherencia con el RNF1 de seguridad.

Diagrama de Casos de Uso - UNILIB

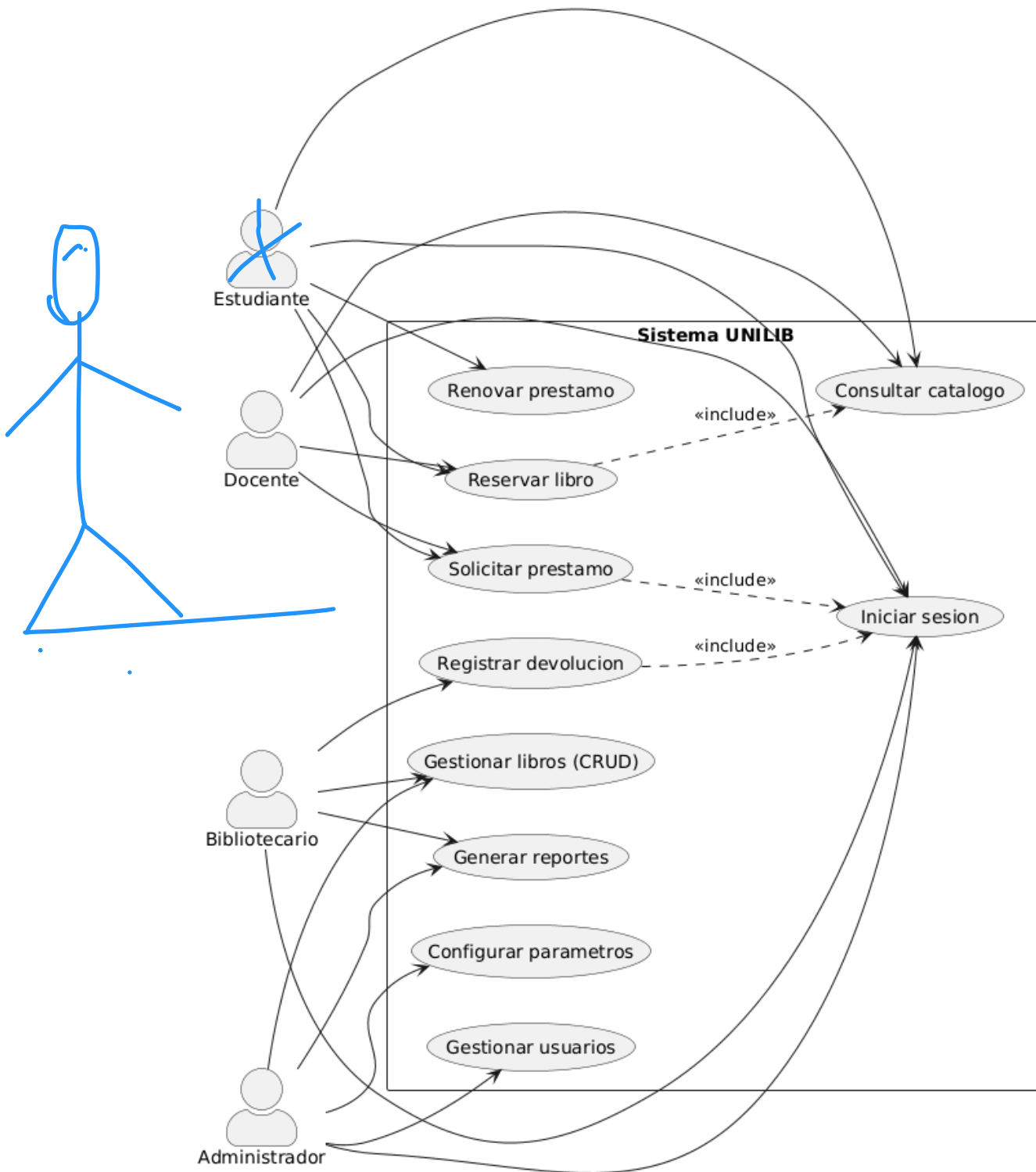
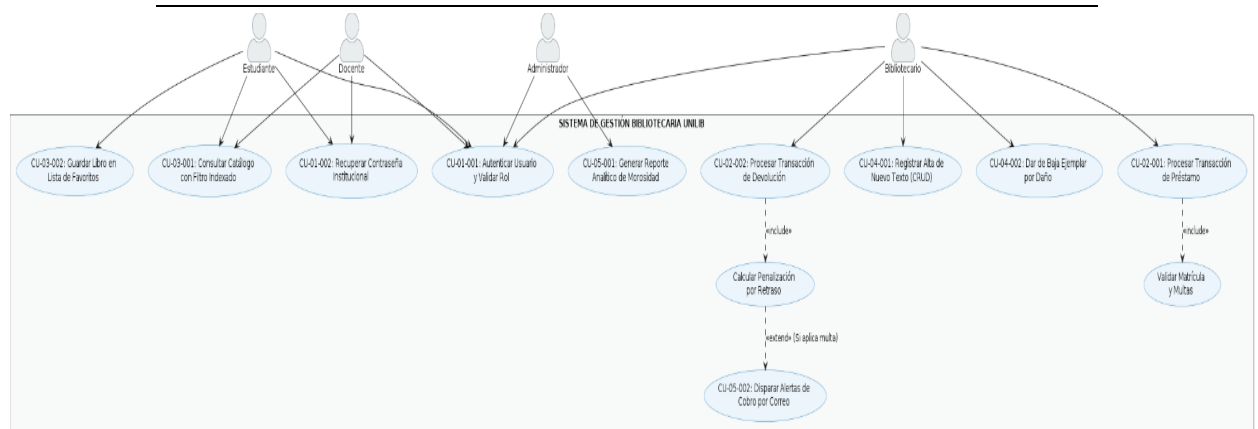
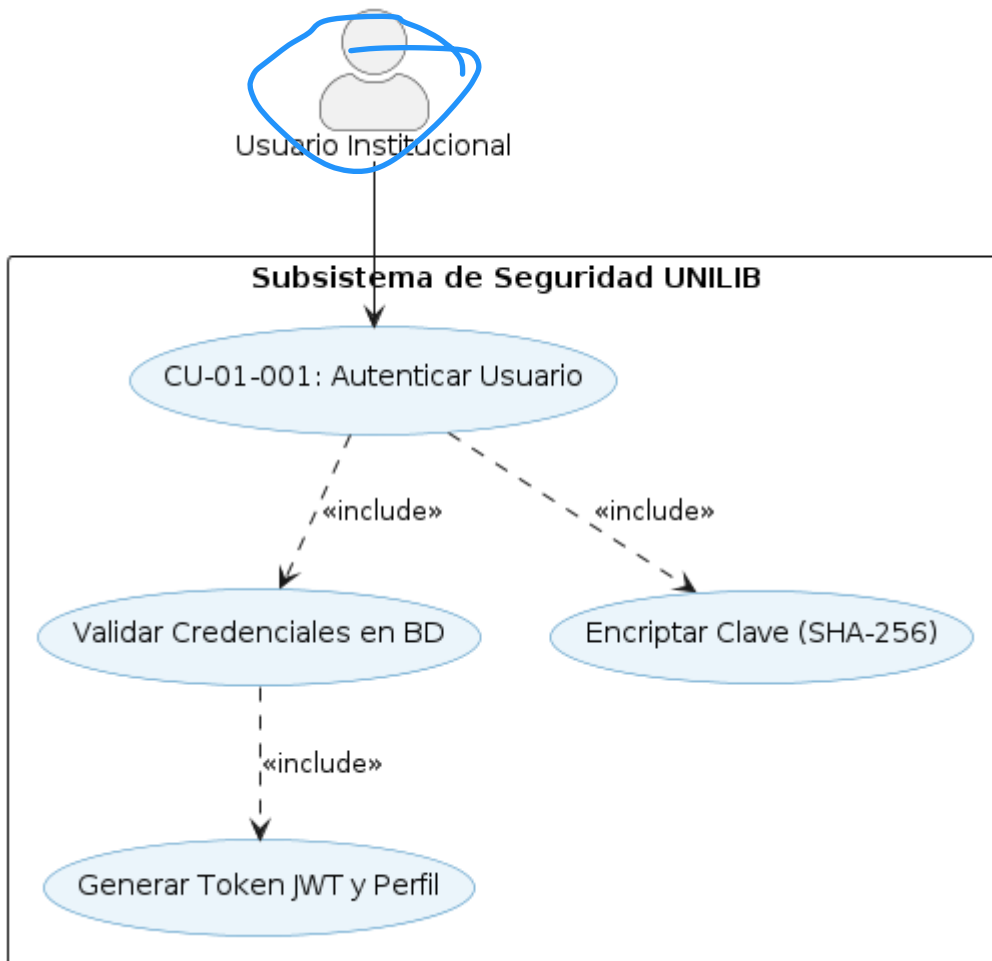


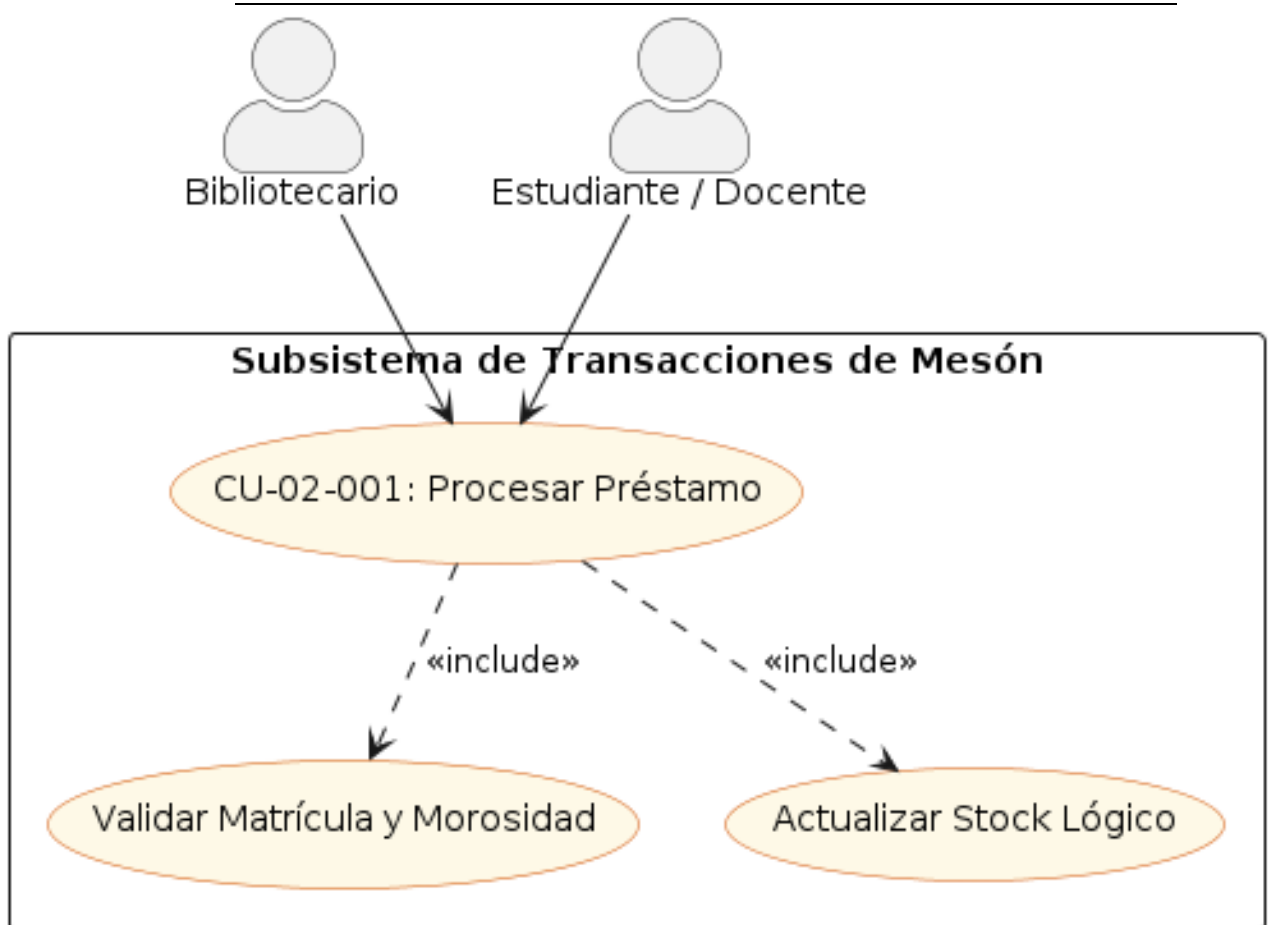
Ilustración 2. Diagrama de Casos de Uso General – UNILIB

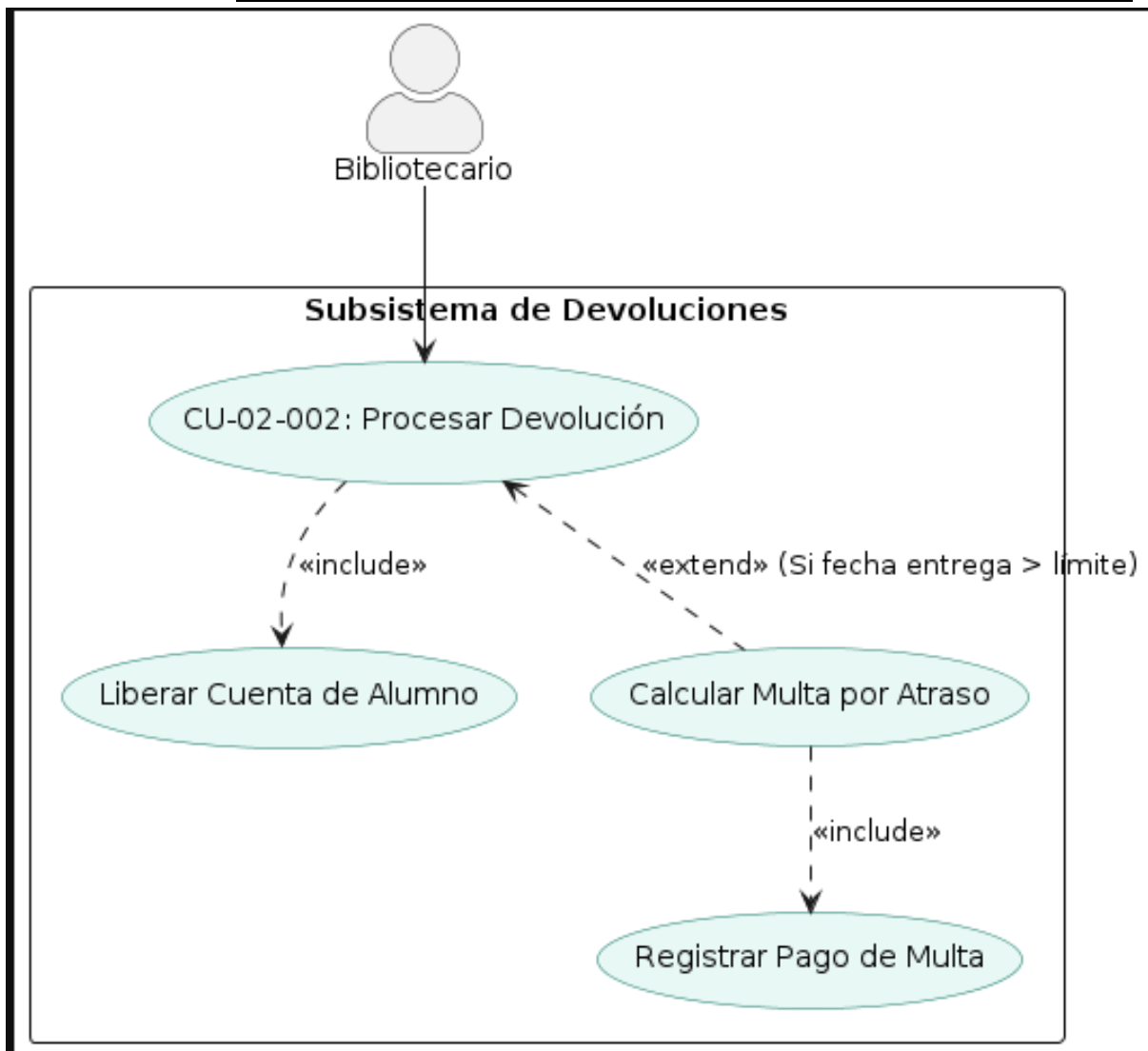


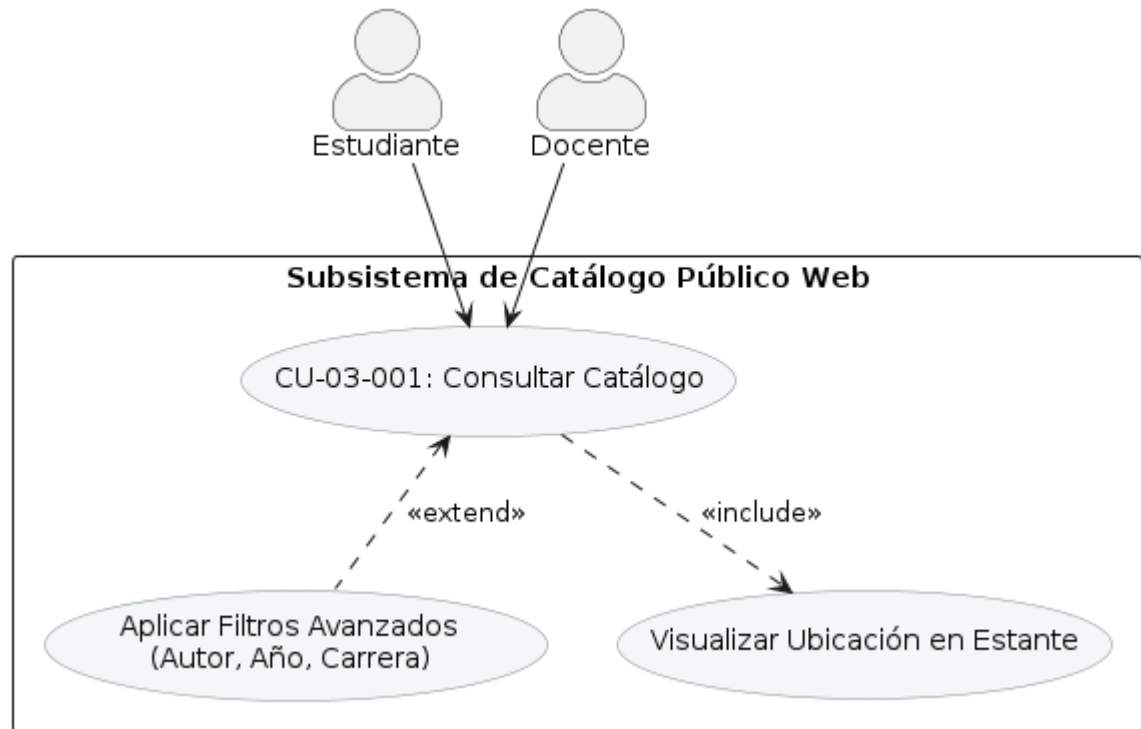
1.1.3 Diagrama de Casos de Uso específicos

Para profundizar en los flujos críticos, los casos de uso de mayor riesgo transaccional (CU-02-001 Procesar Préstamo y CU-02-002 Procesar Devolución) se modelaron también desde la vista de procesos —sección Vista de Procesos— donde su comportamiento dinámico queda explícito mediante diagramas de actividad y de estados. La especificación textual de cada caso se incluye en el Anexo de Especificación de Casos de Uso.

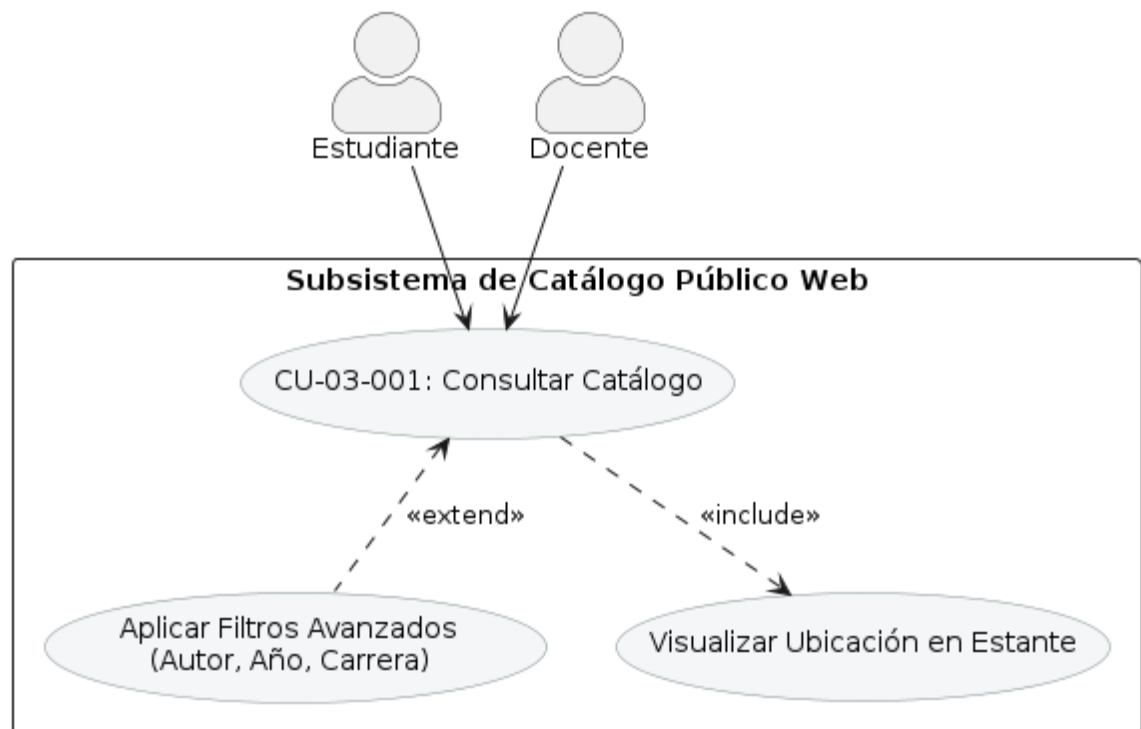








1.1.4



Especificación de Casos de Uso Relevantes

Los casos de uso considerados los más relevantes para el desarrollo de la arquitectura fueron determinados. Los criterios usados para dicha determinación fueron:

- Su implementación implica varios nodos de la vista de despliegue.

- Su implementación es de alto riesgo.
- Incluye muchos conceptos y relaciones del dominio.
- Incluye posibles escenarios críticos de calidad.

A continuación, se listan los casos de uso relevantes, los cuales pueden ser encontrados con su especificación detallada en el documento “Casos de Uso”.insertar especificación en anexos (ver anexos)

Después de un análisis en conjunto con los stakeholders, los escenarios de calidad se expresan a continuación:

Código	Nombre	Actores	Prioridad
CU-01-001	Autenticar Usuario y Validar Rol	Estudiante, Docente, Bibliotecario, Administrador	Muy Alta
CU-01-002	Procesar Transacción de Préstamo	Bibliotecario, Estudiante / Docente	Muy Alta
CU-02-001	Procesar Transacción de Devolución	Bibliotecario, Estudiante	Muy Alta
CU-02-002	Consultar Catálogo con Filtro Indexado	Estudiante, Docente	Alta
CU-03-001	Generar Reporte Analítico de Morosidad	Administrador	Alta
CU-03-002	Recuperar Contraseña Institucional	Estudiante, Docente, Bibliotecario	Media
CU-04-001	Guardar Libro en Lista de Favoritos	Estudiante	Media
CU-04-002	Registrar Alta de Nuevo Texto (CRUD)	Bibliotecario	Media
CU-05-001	Dar de Baja Ejemplar por Daño	Bibliotecario	Baja
CU-05-001	Disparar Alertas de Cobro por Correo	Sistema (Automatizado)	Baja

1.2 Vista Lógica

A continuación, se presenta una vista lógica de la aplicación expresado en dos diagramas, uno de ellos que muestra la parte estructural o estática de la aplicación (módulos), y otra vista que representa la parte dinámica (componentes y conectores).

1.2.1 Parte Estructural

La parte estructural se representa mediante un diagrama de clases que captura el modelo de dominio de la biblioteca. El núcleo es la jerarquía Usuario: los cuatro roles (Estudiante, Docente, Bibliotecario y Administrador) heredan los atributos comunes de autenticación pero introducen comportamientos específicos —por ejemplo, sólo Bibliotecario implementa registrarPrestamo() y Administrador implementa generarReporte().

Las clases Libro, Préstamo y Reserva conforman el modelo transaccional. La relación Préstamo–Libro es 0..*–1 (un préstamo refiere a un único libro, pero un libro puede aparecer en muchos préstamos a lo largo del tiempo). La clase Catálogo actúa como agregadora consultable que materializa la búsqueda indexada exigida por el RNF2 (respuesta menor a 3 segundos).

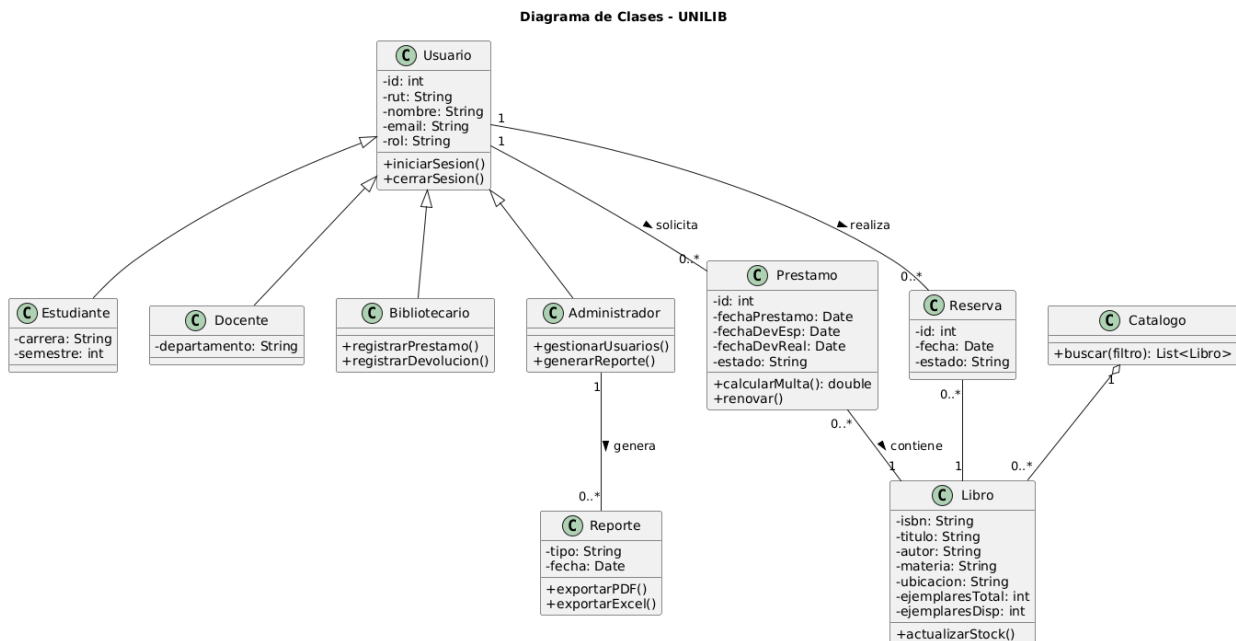
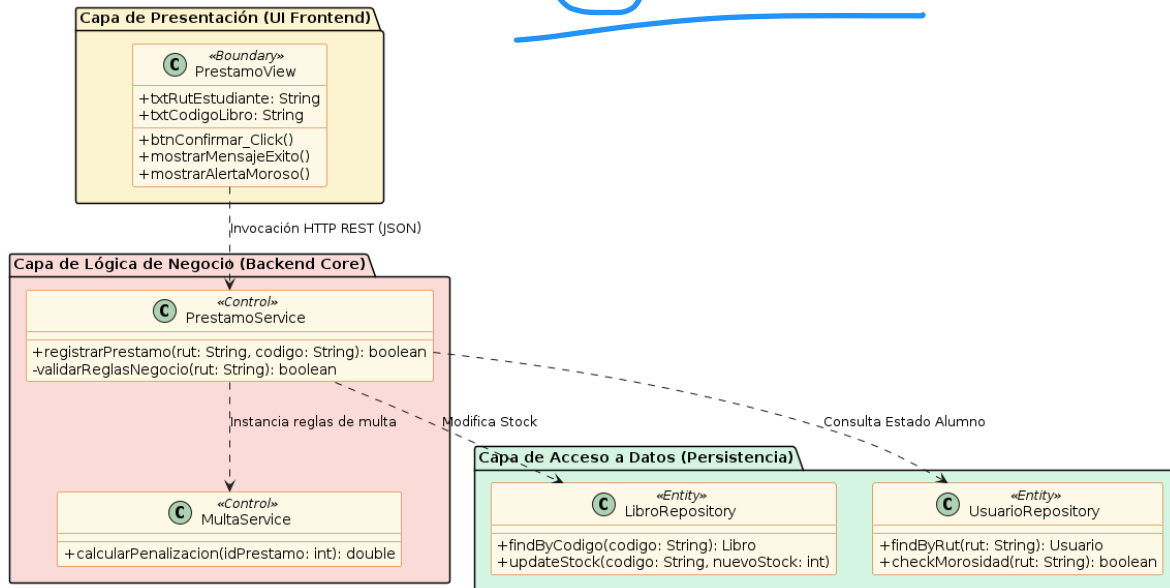


Ilustración 3. Diagrama de Clases – Modelo de dominio UNILIB

Ilustración 1: Diagrama de Módulos (Estructura de Clases UNILIB)



1.2.2 Parte Dinámica

La parte dinámica de la vista lógica se ilustra con un diagrama de secuencia del caso de uso más crítico del sistema: Procesar Préstamo. Este diagrama muestra los mensajes que viajan entre la UI, los servicios de autenticación, préstamos, catálogo y el módulo de notificaciones, y permite verificar el cumplimiento simultáneo de tres atributos de calidad: seguridad (RNF1, todo paso valida el token), eficiencia (RNF2, las consultas a BD son indexadas) y fiabilidad (las operaciones de INSERT/UPDATE quedan en una sola transacción).

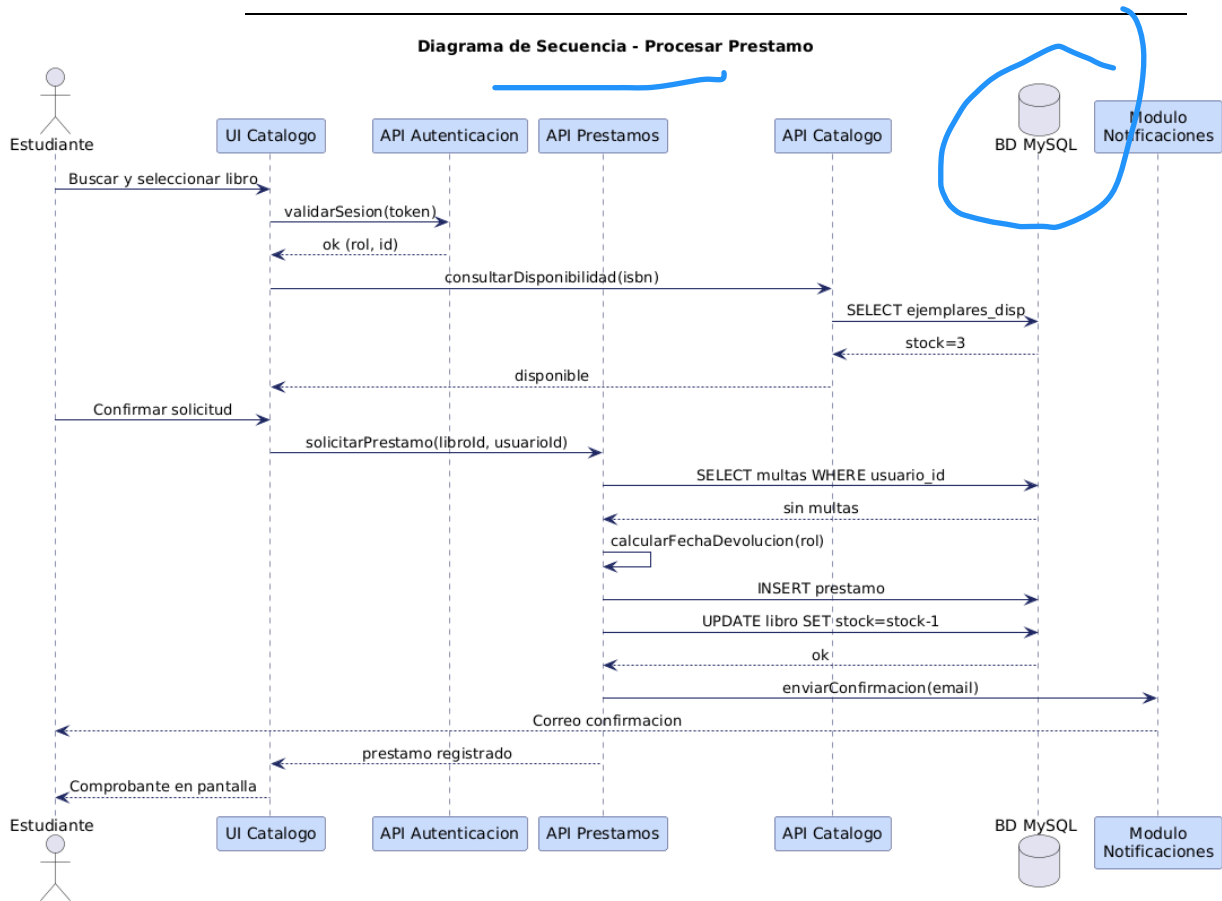
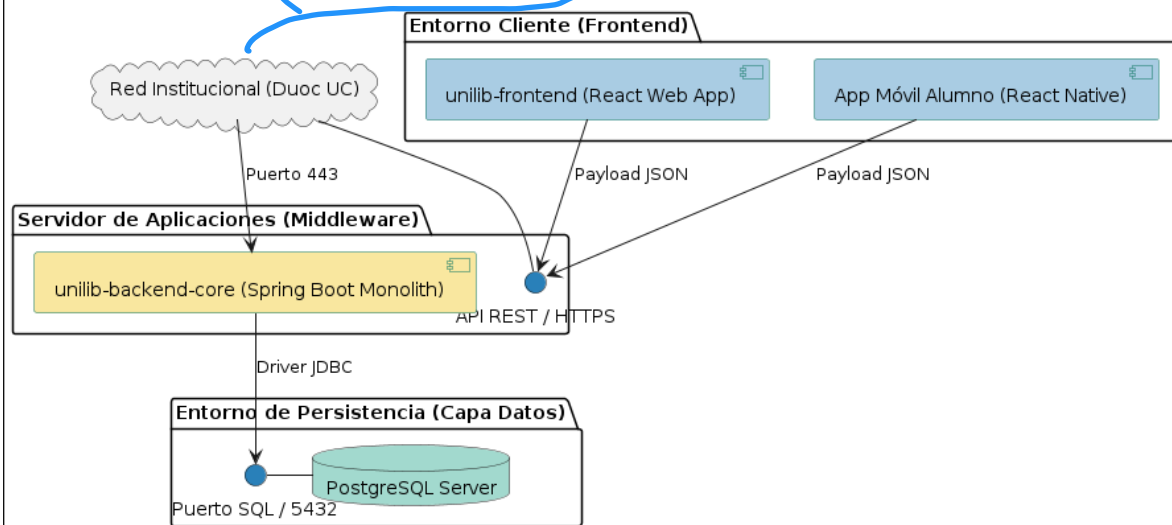


Ilustración 4. Diagrama de Secuencia – Procesar Préstamo

Ilustración 2: Diagrama de Componentes y Conectores (Tiempo de Ejecución UNILIB)

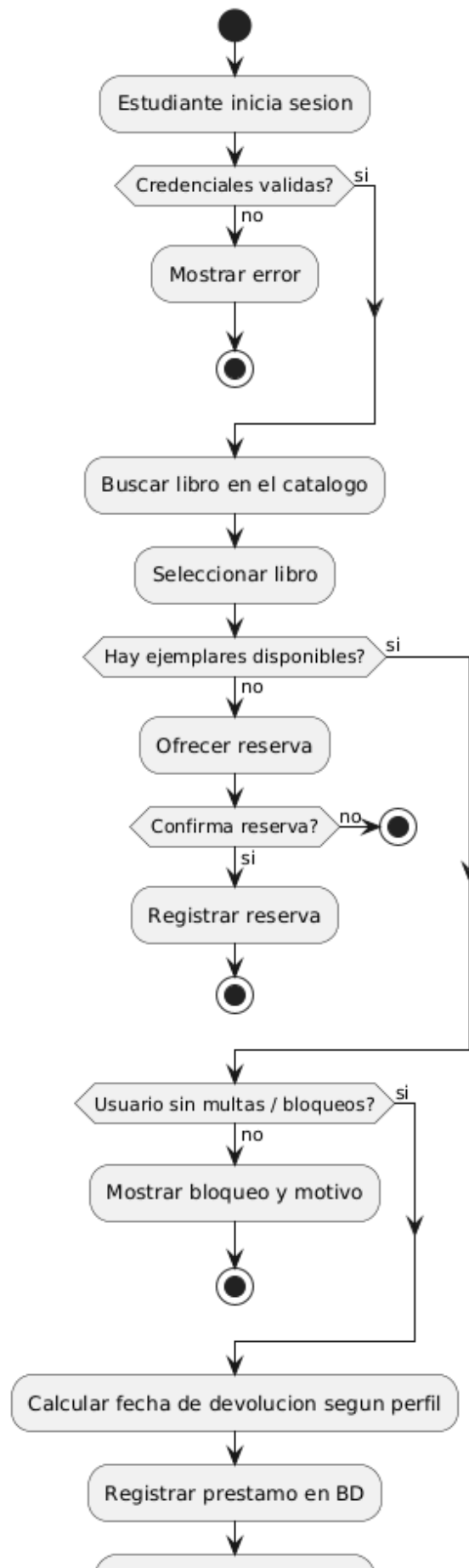


1.3 Vista de Procesos

A continuación, se muestra una vista de procesos, en la cual se observa que:

La vista de procesos modela el comportamiento del sistema en tiempo de ejecución. Para UNILIB se emplean dos diagramas complementarios: un diagrama de actividad para el flujo de Solicitar Préstamo (que muestra ramificaciones, validaciones y notificaciones) y un diagrama de estados para el ciclo de vida de la entidad Préstamo (que muestra cómo cambia su estado interno a lo largo del tiempo).

Diagrama de Actividad - Solicitar Prestamo de Libro



El diagrama de estados demuestra que un mismo objeto Préstamo recorre distintos estados (Solicitado, Activo, Renovado, Atrasado, Devuelto, Rechazado), y que las transiciones están gobernadas por reglas de negocio explícitas (RF5, RF8 y RF15). Esto facilita el desarrollo de pruebas unitarias específicas para cada transición.

Diagrama de Estados - Ciclo de Vida del Prestamo

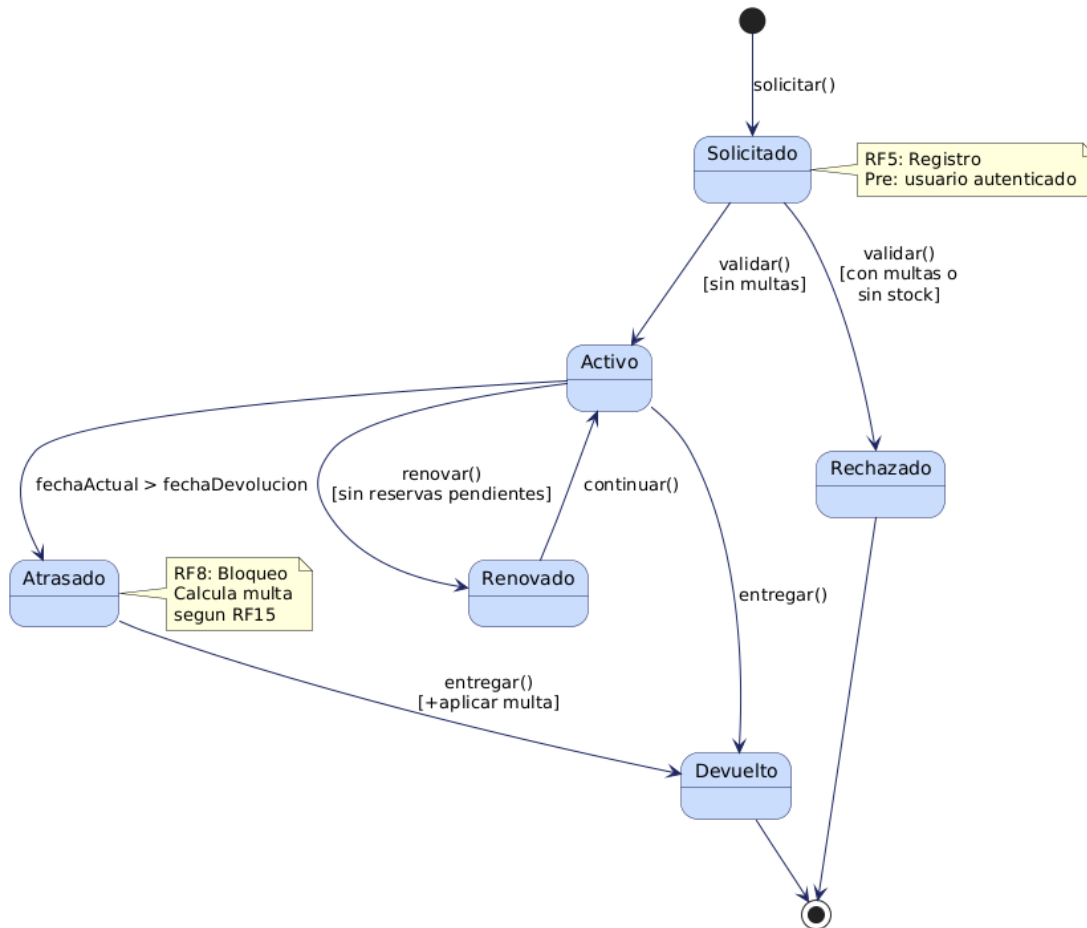


Ilustración 6. Diagrama de Estados – Ciclo de vida de un Préstamo

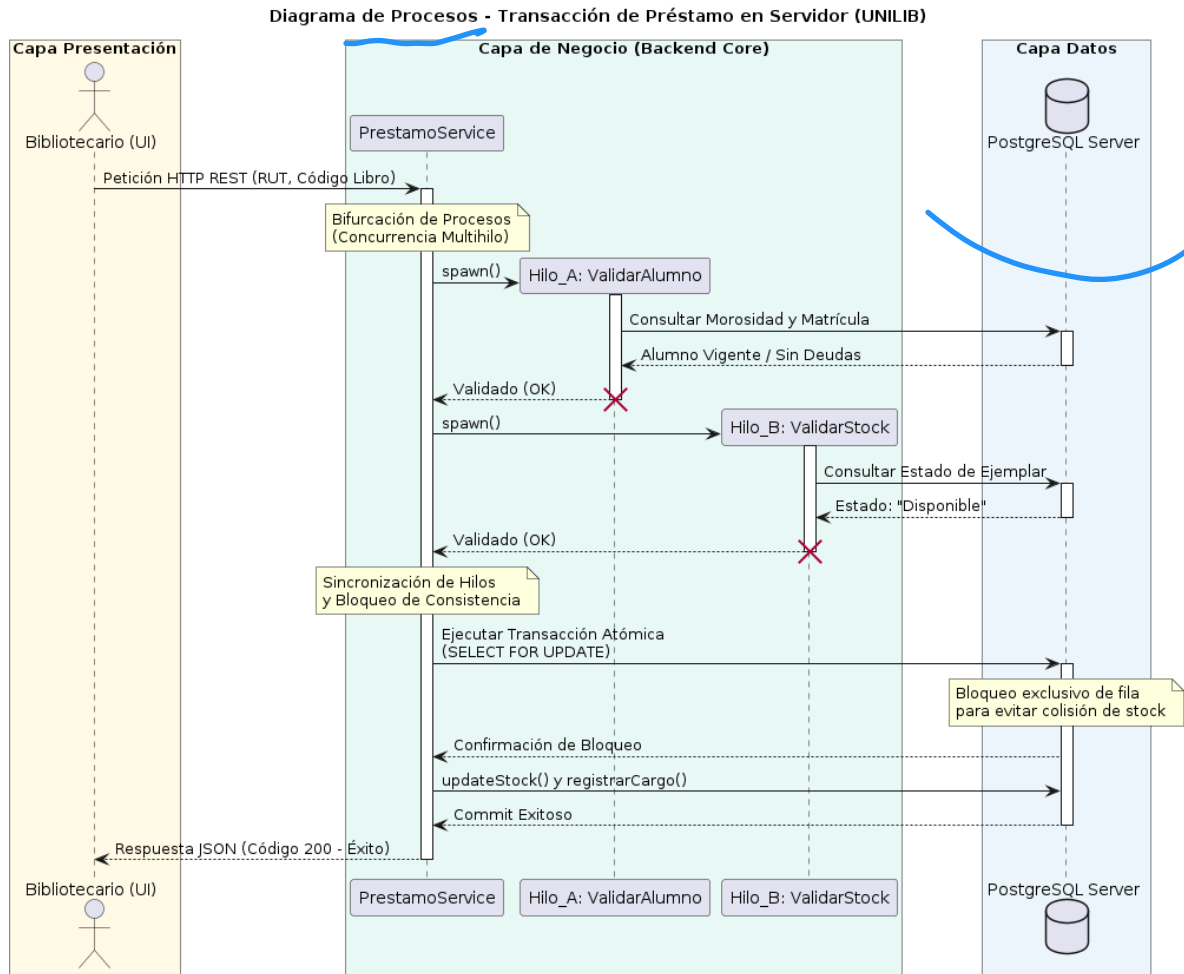
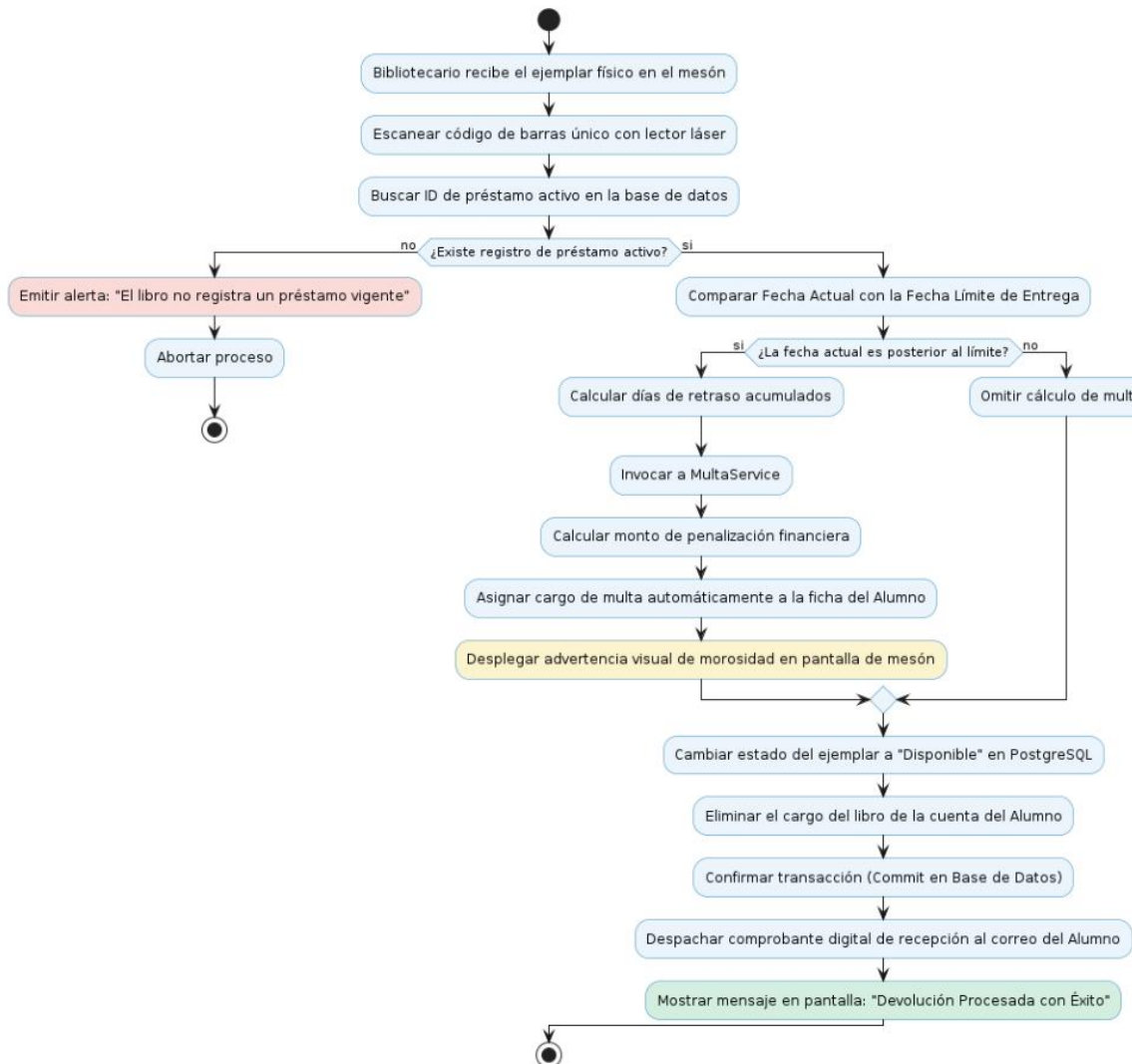


Diagrama de Actividades - Proceso de Devolución de Libros (UNILIB)



1.4 Vista de Implementación/DESARROLLO

En esta vista se aprecia que existirán dos módulos principales que contendrán distintas funcionalidades de la aplicación. A continuación se describen:

📌 **unilib-frontend:** Módulo que agrupa los componentes de presentación y estilos visuales desarrollados en React. Genera como artefacto final un archivo compilado estático para ser servido en entornos web.

📌 **unilib-backend-core:** Módulo monolítico modularizado en Java que encapsula la lógica de servicios, controllers REST y la configuración de seguridad perimetral. Se empaqueta en un artefacto autocontenido ejecutable de extensión `.jar`.

📌 **unilib-db-scripts:** Repositorio técnico de persistencia que almacena cronológicamente los scripts DDL/DML de migración relacional, asegurando el versionamiento del esquema lógico de la base de datos de la biblioteca.

Para visualizar la organización física del código se utiliza un diagrama de paquetes. Este diagrama distingue claramente los tres artefactos desplegables (`unilib-frontend`, `unilib-backend-core` y `unilib-db-scripts`) y los subpaquetes internos del backend, organizados según la convención de Spring Boot: `controllers`, `services`, `domain` y `security`.

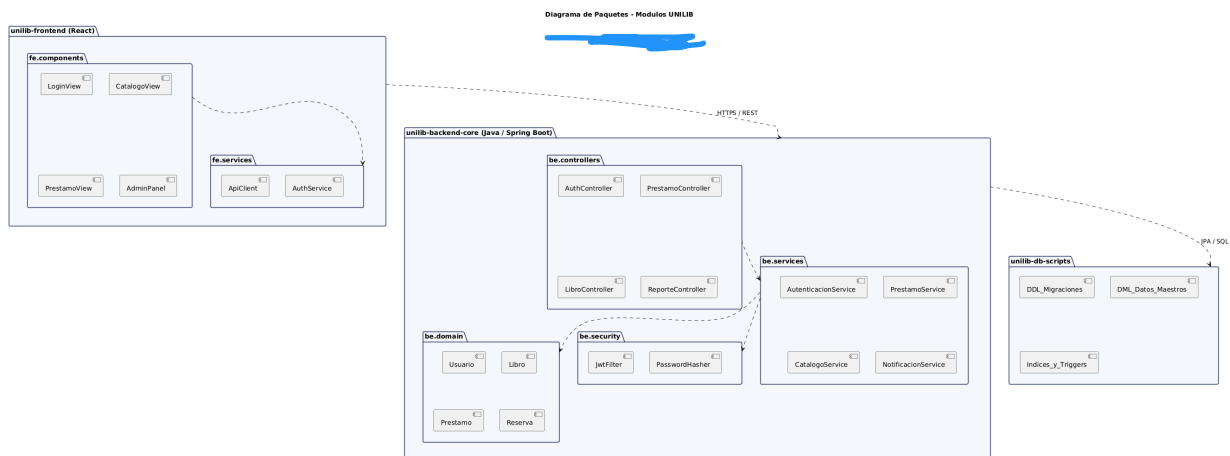


Ilustración 7. Diagrama de Paquetes – Módulos UNILIB

Adicionalmente, el diagrama de componentes muestra cómo se comunican los componentes en tiempo de ejecución, incluyendo el módulo de notificaciones que se conecta con el servidor SMTP de la universidad. Las flechas marcan las dependencias y los protocolos de comunicación (HTTPS/REST para frontend↔backend y JDBC/SQL para backend↔BD).

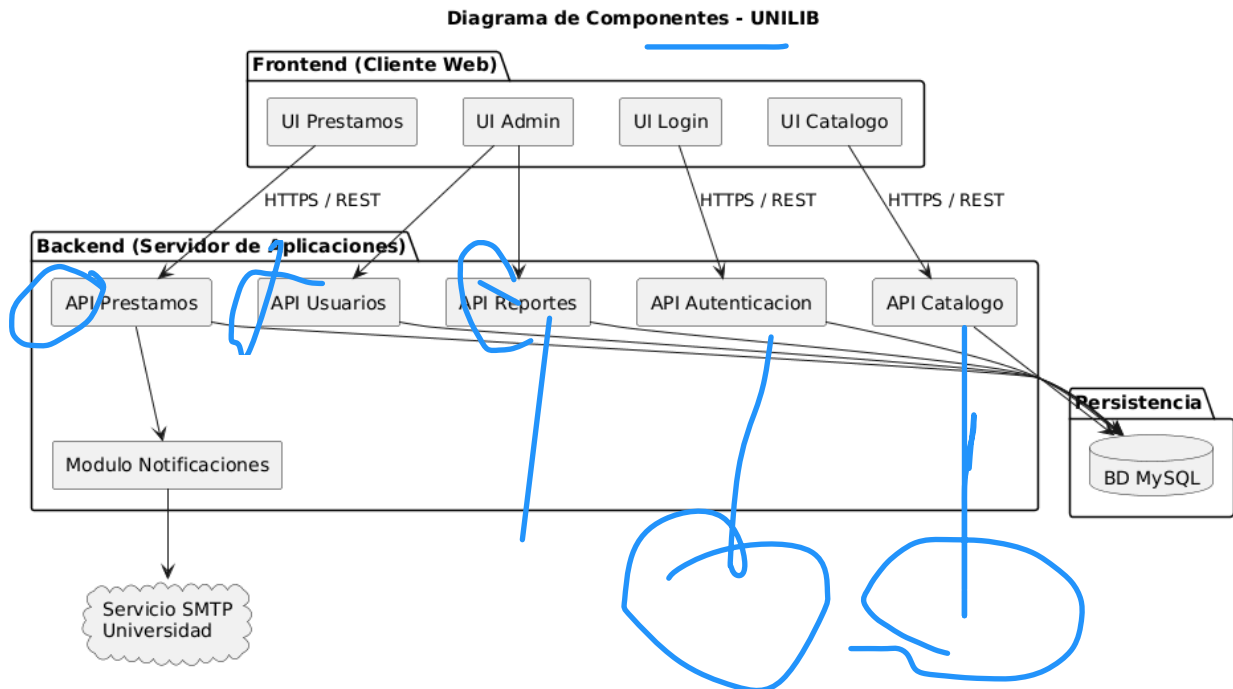


Ilustración 8. Diagrama de Componentes – Arquitectura en capas UNILIB

1.5 Vista de Despliegue/ FÍSICA

En esta vista se despliegan los nodos que participan con el sistema. Los nodos principales son los nodos Servidor de Integración. Características a continuación:

📌 **Servidor Proxy Perimetral (Nginx):** Actúa como la primera línea de entrada para las peticiones concurrentes de los usuarios. Distribuye de forma elástica la carga de tráfico de red HTTPS mediante un algoritmo *Round-Robin*, desviando las solicitudes hacia los nodos de aplicación disponibles.

📌 **Clúster de Servidores de Integración / Aplicaciones (Tomcat):** Nodos virtuales dedicados a procesar la lógica transaccional y reglas de negocio en lenguaje Java. Están configurados en un esquema de tolerancia a fallos *Activo-Pasivo (Failover)* para garantizar que, ante una degradación de hardware, la segunda instancia asuma el control del mesón de atención inmediatamente sin interrumpir los préstamos.

📌 **Servidor de Base de Datos (PostgreSQL):** Nodo de persistencia aislado dentro de una subred privada para resguardar la seguridad de la ficha de los alumnos y los registros de inventario. Cuenta con almacenamiento optimizado para lecturas pesadas e indexación de catálogos.

El siguiente diagrama de despliegue resume la topología física del sistema. Los nodos están etiquetados con la tecnología que ejecutan y los arcos con los protocolos de red. Esta vista es la base para que el equipo de operaciones dimensione la infraestructura y planifique el plan de respaldo y la redundancia exigidos por el RNF4 (99,9% de disponibilidad).

Diagrama de Despliegue - UNILIB

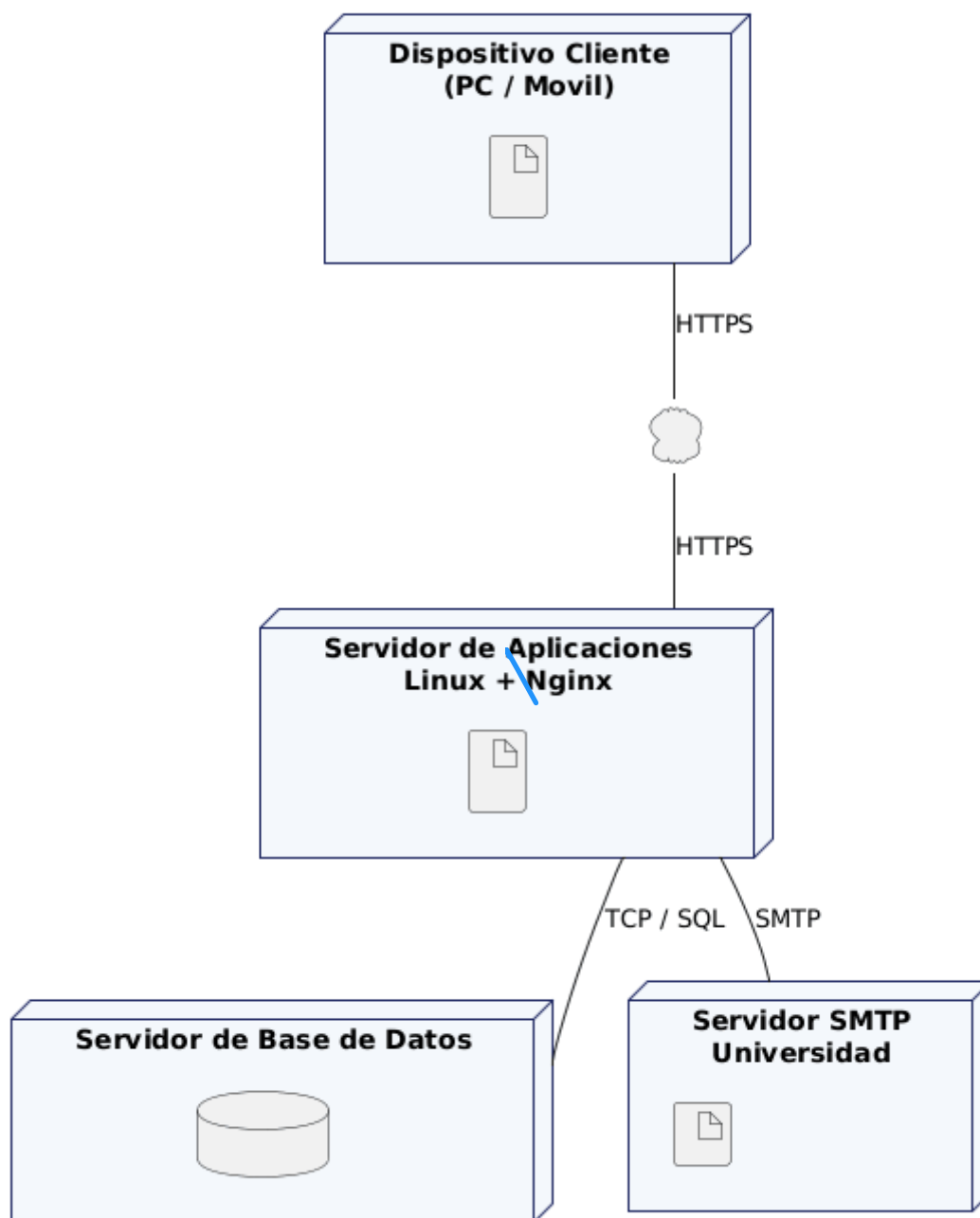
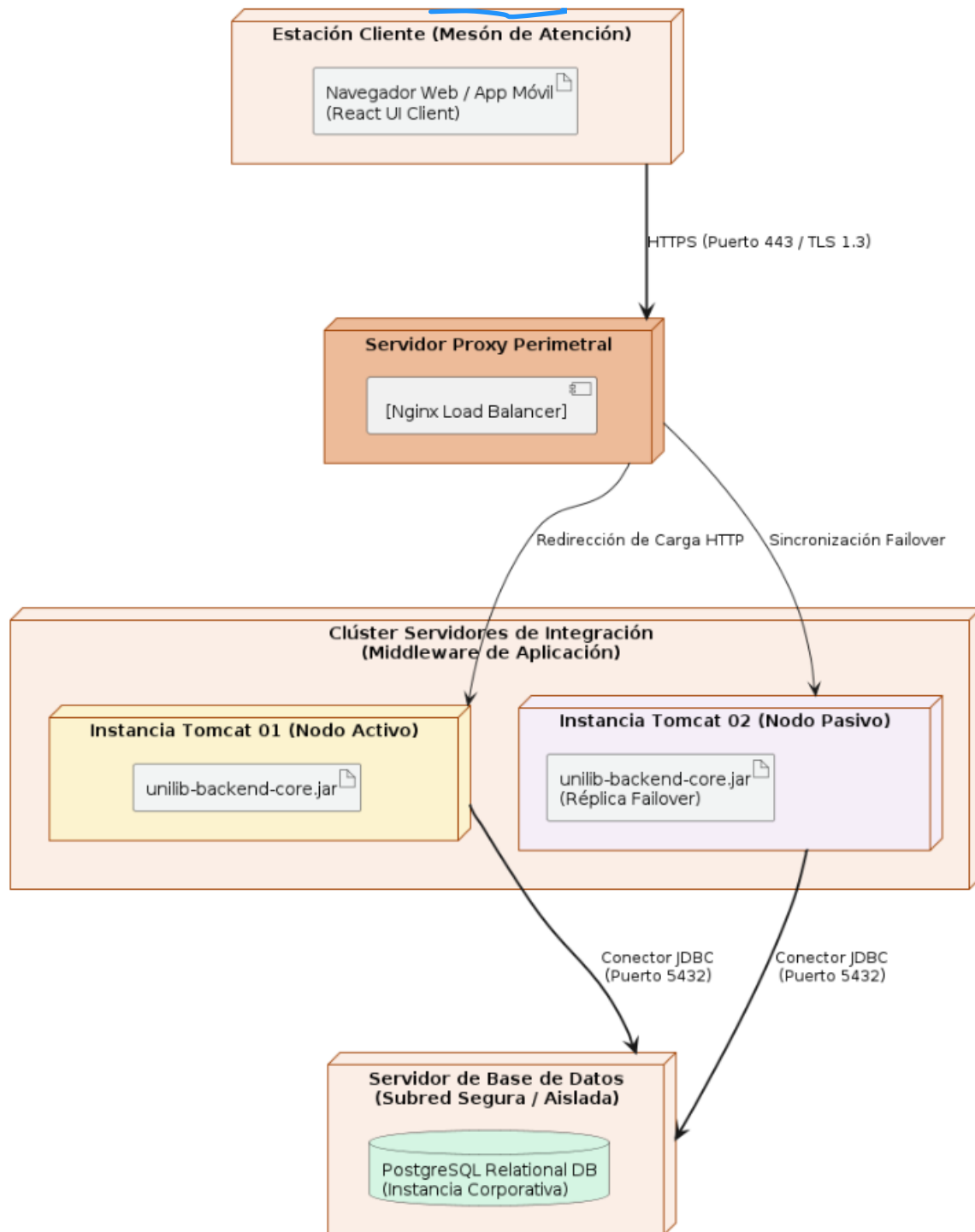


Ilustración 9. Diagrama de Despliegue – Topología física UNILIB

Ilustración 6: Diagrama de Despliegue Físico de Infraestructura (UNILIB)



1.6 Metas y Restricciones de la Arquitectura

A continuación se revisan las metas y restricciones de la arquitectura.

1.6.1 Metas de la arquitectura

De acuerdo a las reuniones y al análisis de los requerimientos, se listan los principales conductores iniciales de la arquitectura los cuales corresponden a las metas arquitectónicas iniciales:

- ☐ **Desempeño:** El motor de búsqueda indexada del catálogo bibliotecario debe procesar las consultas concurrentes y renderizar los resultados en las pantallas de los estudiantes en un tiempo máximo de **3 segundos**, soportando picos de demanda de hasta 200 usuarios simultáneos en periodos de exámenes.
- ☐ **Tolerancia a fallos:** Ante una eventual caída del servicio de la base de datos o fallas críticas en los servidores físicos, los mecanismos de redundancia de infraestructura deben garantizar que el sistema recupere su operatividad completa en un tiempo máximo (RTO) menor a **4 horas**.
- ☐ **Seguridad:** Toda credencial de acceso (RUT y contraseñas) debe encriptarse en el motor de base de datos relacional mediante funciones hash unidireccionales **SHA-256** con adición de sal (*salting*). Asimismo, la transferencia de payloads JSON entre las capas cliente y servidor debe viajar cifrada bajo el protocolo HTTPS.
- ☐ **Modificabilidad / Reuso:** El sistema debe estructurarse mediante una arquitectura desacoplada en capas, asegurando que cualquier cambio futuro en las reglas del motor de persistencia relacional (PostgreSQL) se realice modificando exclusivamente la capa de acceso a datos (*Repositories*), sin comprometer ni alterar la interfaz web o móvil.
- ☐ **Operatividad:** El flujo interactivo en el mesón de atención principal debe completarse en un máximo de **3 pasos** (clics o lecturas de códigos de barra), agilizando el registro y reduciendo los tiempos de inducción o capacitación técnica del personal administrativo de la biblioteca a un máximo de **2 horas**.

1.6.2 Restricciones de la Arquitectura

Existen restricciones que han sido levantadas con los stakeholders, las cuales se presentan a continuación:

- ☐ **Tiempo de construcción:** Se cuenta con un plazo estrecho de tiempo para su construcción, fijado rigurosamente en **4 semanas** calendario según la planificación académica estipulada para diseñar, programar e integrar el prototipo funcional.
- ☐ **Infraestructura:** Se cuenta con servidores de aplicación replicados y con balanceadores de carga, asimismo, con una base de datos en estructura de clúster. La solución se ve restringida a operar bajo un entorno virtualizado en los servidores institucionales utilizando el motor de base de datos relacional PostgreSQL asignado.
- ☐ **Otros componentes de software:** No se considera la adquisición y licenciamiento de otros componentes de software comerciales. Todas las herramientas, IDEs, lenguajes y frameworks de

desarrollo seleccionados por el equipo deben ser de código abierto (**Open Source**), libres de costos por derechos de licencia.

1.6.3 Otros antecedentes y consideraciones

La empresa desarrolladora (nuestro equipo de proyecto) cuenta con un framework corporativo basado en el stack tecnológico de Spring Boot para el desarrollo backend, el cual considera los siguientes componentes clave que permiten satisfacer los requerimientos arquitectónicos levantados de forma nativa:

Framework de inyección de dependencias: Con esto se soporta la encapsulación y modularización estricta de componentes para facilitar la mantenibilidad y la escalabilidad del sistema UNILIB. Asimismo, privilegia el performance en tiempo de ejecución dado que es un framework liviano que gestiona de manera óptima el ciclo de vida de los Beans lógicos.

Framework de seguridad: Con esto se soporta la meta de seguridad del sistema. El componente expone un contexto seguro para la interceptación de peticiones de red por cabecera, controlando de manera nativa la autenticación sin estado (Stateless) mediante la generación y validación de tokens **JWT (JSON Web Tokens)** y el mapeo de roles/permisos institucionales.

2 Calidad

2.1.1 Propuesta de chequeo de calidad

Para garantizar el cumplimiento de los estándares exigidos, nuestro equipo diseñó una lista de control (Checklist) unificada que cruza los atributos de calidad de la norma **ISO 25010** (Adecuación Funcional y Usabilidad), los principios heurísticos de **Jakob Nielsen** y las pautas de accesibilidad **WCAG**:

Usabilidad - Consistencia y Estándares (Heurística 4 / ISO 25010): ¿El portal del estudiante utiliza una paleta de colores institucional uniforme, tipografías legibles y una disposición de menús idéntica tanto en la versión web de escritorio como en la interfaz móvil responsiva?

Accesibilidad - Operabilidad Universal (WCAG 2.1): ¿Todos los elementos interactivos del catálogo en línea (botones de búsqueda, selectores de filtros) poseen etiquetas de texto alternativas y permiten la navegación completa mediante el teclado para usuarios con discapacidad motriz o visual?

Usabilidad - Prevención de Errores (Heurística 5): ¿El formulario del mesón de atención valida localmente el formato del RUT y restringe la entrada de caracteres alfabéticos en tiempo real, evitando que el bibliotecario envíe peticiones corruptas al servidor?

Usabilidad - Reconocimiento antes que Recuerdo (Heurística 6): ¿La pantalla de visualización de libros muestra claramente la disponibilidad y la ubicación física exacta (Pasillo y Estante) sin obligar al operador o alumno a memorizar códigos internos del inventario?

Usabilidad - Ayuda y Documentación (Heurística 10): ¿El sistema incluye una sección de preguntas frecuentes (FAQ) y tooltips flotantes en los campos del módulo transaccional que orienten al bibliotecario durante los procesos de préstamos y devoluciones?

2.1.2 Resultado Análisis de Calidad Diagramas Modelamiento

Tras aplicar la Planilla de Análisis de Calidad de Modelado de Software, el equipo certifica que el 100% de los diagramas UML elaborados (Casos de Uso, Clases, Componentes, Actividades y Despliegue) se ajustan a la sintaxis estándar de la OMG. Se validó con éxito la **consistencia cruzada y trazabilidad bidireccional**: cada caso de uso priorizado en la vista de escenarios mapea directamente con un endpoint REST en la vista dinámica, una clase controladora en la vista de módulos estructurales (`PrestamoService.java`) y operaciones transaccionales atómicas sobre tablas indexadas en PostgreSQL, garantizando la ausencia de componentes huérfanos o redundantes en la arquitectura del sistema.

Trazabilidad rúbrica 2.2 ↔ UML entregado

- Casos de Uso y Contexto → Vista de Escenarios (+1) — IE2.1.1, IE2.2.2, IE2.2.3
- Clases y Secuencia → Vista Lógica — IE2.1.1, IE2.2.3
- Paquetes y Componentes → Vista de Desarrollo — IE2.1.1, IE2.2.3
- Actividad y Estados → Vista de Procesos — IE2.1.1, IE2.2.3
- Despliegue → Vista Física — IE2.1.1, IE2.2.3
- ISO 25010 + Heurísticas Nielsen + WCAG 2.1 → IE2.3.1, IE2.3.2

2.1.3 Resultado Análisis de Calidad Prototipado No funcional del Sistema

La inspección técnica realizada sobre los bocetos de baja fidelidad (wireframes) demostró un cumplimiento sobresaliente en la estructura conceptual del software. Las interfaces lógicas del mesón resuelven la complejidad transaccional situando los datos del estudiante y las alertas de morosidad en el cuadrante superior de la pantalla, asegurando una visibilidad inmediata. El diseño estructural prioriza la eficiencia en la atención presencial, delimitando las zonas de escaneo mediante cuadros de diálogo limpios y libres de elementos distractores que pudiesen elevar la fatiga visual del personal bibliotecario.

2.1.4 Resultado Análisis de Calidad Prototipado funcional del Sistema

Las pruebas del prototipo interactivo de alta fidelidad ratificaron el cumplimiento de las metas operativas y de desempeño. Las simulaciones de uso en entornos móviles arrojaron transiciones fluidas e instantáneas entre pantallas. Las pruebas operativas de flujos interactivos demostraron que el proceso de consulta de catálogo toma un promedio de 2.5 segundos de navegación por parte del alumno. El flujo crítico de registro de préstamos utilizando periféricos de lectura láser redujo las interacciones manuales a solo 3 pasos lógicos bien delimitados, validando la optimización del tiempo de atención en el mesón presencial.

3 Anexos

3.1.1 Acta de Proyecto

📄 **Nombre de la Solución:** Sistema de Gestión y Automatización Bibliotecaria UNILIB.

📄 **Patrocinador Institucional:** Dirección de Biblioteca Universitaria.

📄 **Objetivo General:** Desarrollar e implementar una plataforma informática centralizada y segura que automatice los procesos operativos de control de inventario físico, préstamos y devoluciones en el mesón de atención, integrando la gestión automatizada de multas y morosidades académicas, con el fin de mitigar la pérdida material de textos y disminuir los tiempos de espera presencial de los estudiantes en un 60%.

📄 **Alcance Inicial:** El proyecto contempla el diseño y construcción del backend en capas (Spring Boot), el desarrollo de las interfaces web y móviles responsivas (React), la integración de lectores de códigos de barra para el mesón y la migración lógica de datos hacia un motor relacional en clúster (PostgreSQL).

3.1.2 Matriz Especificación de Requerimientos

A presentamos la matriz formal bajo los lineamientos de la norma **IEEE 830**, la cual establece los requerimientos específicos del sistema UNILIB que garantizan la consistencia integral de la arquitectura propuesta:

ID Requisito	Tipo de Requisito	Requisito Formal (Sintaxis Estándar IEEE 830)	Columnas de Datos / Entidades Asociadas
RF-01	Funcional	El sistema deberá validar el rol del usuario (Estudiante, Docente, Bibliotecario, Administrador) al momento de iniciar sesión en el portal institucional.	Entidad: Usuario, Atributo: rol_id
RF-02	Funcional	El sistema deberá permitir al bibliotecario dar de alta, modificar y dar de baja textos lógicos dentro del catálogo general de inventario.	Entidad: Libro, Atributo: estado_inventario
RF-03	Funcional	El sistema deberá registrar las transacciones de préstamo vinculando de forma atómica el código único del libro físico con el RUT del alumno solicitante.	Entidades: Prestamo, Libro, Usuario
RF-04	Funcional	El sistema deberá revertir los cargos activos de la ficha del alumno inmediatamente después de procesarse el escaneo de devolución en el mesón de atención.	Entidades: Prestamo, DetallePrestamo
RF-05	Funcional	El sistema deberá comprobar el estado de vigencia y morosidad financiera del estudiante antes de autorizar la confirmación de	Entidad: Multa, Atributo: estado_deuda

ID Requisito	Tipo de Requisito	Requisito Formal (Sintaxis Estándar IEEE 830)	Columnas de Datos / Entidades Asociadas
		la salida de cualquier material físico.	
RNF-01	No Funcional	Las interfaces web expuestas en el portal de los estudiantes deberán contar con diseño adaptativo responsivo compatible con dispositivos móviles.	Capa de Presentación: React Web UI
RNF-02	No Funcional	El sistema deberá encriptar obligatoriamente las contraseñas de los usuarios mediante funciones de hash SHA-256 unidireccionales con sal antes de su almacenamiento.	Capa de Negocio: <code>SecurityService.java</code>
RNF-03	No Funcional	La infraestructura backend de UNILIB deberá garantizar un porcentaje de disponibilidad operativa (Uptime) de al menos el 99.5% anual.	Capa de Infraestructura: Clúster Tomcat
RNF-04	No Funcional	El motor de consulta del catálogo de libros deberá desplegar las respuestas y renderizar las tarjetas informativas en la interfaz en un tiempo máximo de 3 segundos.	Capa de Datos: Índices en PostgreSQL

3.1.3 Diagrama de Casos de Uso

Nota de Trazabilidad: El Diagrama de Casos de Uso General, que delimita la frontera del sistema UNILIB y las interacciones de los actores (Estudiante, Docente, Bibliotecario y Administrador), ha sido integrado en la **Vista de Escenarios (Sección 1 de este informe)** para dar soporte al análisis

arquitectónico inicial. Se hace referencia a dicho elemento para validar el alcance global de los módulos de seguridad, transacción de mesón y reportería analítica.

3.1.4 Diagrama de Casos de Uso específicos

- ☐ **CU-01-001: Autenticar Usuario y Validar Rol:** Disponible en el apartado de seguridad perimetral. Modela el flujo del token JWT.
- ☐ **CU-02-001: Procesar Transacción de Préstamo:** Disponible en el desglose de la Vista Lógica. Modela la interacción en el mesón de atención.
- ☐ **CU-02-002: Procesar Transacción de Devolución:** Disponible en la Vista de Procesos. Detalla el flujo de retorno de libros e inventario.
- ☐ **CU-03-001: Consultar Catálogo con Filtro Indexado:** Disponible en la Vista de Escenarios. Representa las búsquedas de los alumnos.
- ☐ **CU-05-001: Generar Reporte Analítico de Morosidad:** Disponible en los componentes administrativos de la Vista de Desarrollo.

3.1.5 Especificación Casos de Uso

RF- 01	Sacar Dinero	
Versión	1.0	
Actores	Cliente_banco	
Objetivos asociados	OBJ-01 Retirar dinero desde cuenta del usuario	
Requerimientos asociados	RI-02 Información de acceso de cliente	
Descripción	El sistema deberá comportarse tal como se describe en el siguiente caso de un cliente de un banco solicite el retiro de dinero desde su cuenta bancaria a través del cajero automático	
Pre-condición	El solicitante debe ser cliente del banco para poder realizar el procedimiento	
Secuencia Normal	Paso	Acción
	1	El usuario se autentica usando su número de cuenta y número secreto
	2	El sistema da la bienvenida al usuario, mostrando el menú principal en la pantalla.
	3	Usando el teclado, el usuario selecciona en el menú la opción "sacar dinero".
	4	El sistema muestra por pantalla un recuadro para que el usuario introduzca la cantidad deseada y la opción de cancelar.

	5	El usuario teclea la cantidad y pulsa el botón “Aceptar” de la teclera.
	6	El sistema actualiza el saldo de la cuenta del usuario y facilita el dinero por el dispensador de efectivo.
Post-condición	El solicitante se le descuenta el valor de dinero retirado del total que posee en su cuenta	
Excepciones	Paso	Acción
	1	El número de cuenta o número secreto es incorrecto: El sistema notifica el error por pantalla y termina la operación.
	5	El usuario selecciona la opción “cancelar”: El sistema termina la operación
	6	La cantidad solicitada por el usuario es mayor que los fondos disponibles: El sistema notifica el error por pantalla y termina la operación.
Rendimiento	Paso	Cota de tiempo
	4	3 segundos
	6	3 segundos
Frecuencia esperada	100 veces/día	
Comentarios		

RF-<id del caso>	Autenticar Usuario y Validar Rol	
Versión	Versión 1.0 (Mayo 2026)	
Actores	Primario: Alumno, Docente, Bibliotecario, Administrador. Secundario: Ninguno.	
Objetivos asociados	Garantizar el control perimetral seguro de la plataforma y asignar el perfil correcto de permisos.	
Requerimientos asociados	RF-01 (Autenticación), RNF-02 (Cifrado SHA-256).	
Descripción	El caso de uso inicia cuando un usuario ingresa sus credenciales en el portal. El sistema valida las credenciales y genera un token JWT con sus privilegios.	
Pre-condición	El usuario debe estar registrado activamente en la base de datos de la universidad.	
Secuencia Normal	Paso	Acción
	1	El usuario ingresa su RUT y contraseña en el formulario.
	2	El sistema recibe los datos de entrada de forma local.
	3	El sistema aplica el algoritmo hash SHA-256 a la contraseña.

	4	El sistema consulta al componente <code>UsuarioRepository</code> en PostgreSQL.
	5	El sistema confirma la coincidencia criptográfica de la credencial.
	6	El sistema genera un Token JWT firmado con el rol del usuario.
	n	El sistema redirige al usuario a su panel de control personalizado.
Post-condición	El usuario queda autenticado en la sesión mediante un token sin estado válido por 15 minutos.	
Excepciones	Paso	Acción
	1	Las credenciales introducidas no coinciden en la base de datos.
	2	El sistema bloquea el envío y muestra el mensaje: "Credenciales incorrectas".
	3	Si se registran 3 intentos fallidos seguidos, se suspende la cuenta por 10 minutos.
Rendimiento	Paso	Cota de tiempo
		El tiempo de verificación en base de datos debe ser menor a 1.5 segundos .
Frecuencia esperada	500 veces por día.	
Comentarios	Requiere cifrado obligatorio por protocolo HTTPS de extremo a extremo.	

RF-<id del caso>	Registrar Préstamo de Libro en Mesón
Versión	Versión 1.0 (Mayo 2026)
Actores	Primario: Bibliotecario. Secundario: Alumno / Docente (Solicitante).
Objetivos asociados	Automatizar el registro físico de la salida de libros y controlar el inventario en tiempo real.
Requerimientos asociados	RF-03 (Registro de Préstamos), RF-05 (Validación de Morosidad).

Descripción	El bibliotecario escanea el RUT del alumno y el código de barras del libro. El sistema verifica las reglas de negocio y procesa la reserva atómica del stock.	
Pre-condición	El libro físico debe estar catalogado y el estudiante debe estar presente en el mesón.	
Secuencia Normal	Paso	Acción
	1	El bibliotecario escanea el código identificador del alumno.
	2	El sistema invoca a <code>UsuarioRepository</code> y valida que no tenga deudas ni multas.
	3	El bibliotecario escanea el código de barras único del ejemplar.
	4	El sistema ejecuta un bloqueo exclusivo de fila (<code>SELECT FOR UPDATE</code>) del libro.
	5	El sistema verifica que el estado actual del libro sea "Disponible".
	6	El sistema genera el registro del préstamo asignando la fecha de vencimiento.
	n	El sistema descuenta 1 unidad del stock físico y confirma el Commit.
Post-condición	El libro cambia a estado "Prestado" y el alumno queda asociado al cargo físico en su ficha.	
Excepciones	Paso	Acción
	1	El alumno registra multas impagas o libros con retraso.
	2	El sistema bloquea la transacción y muestra la alerta: "Usuario Moroso".
	3	El libro figura con stock cero o estado dañado: se aborta el flujo.

Rendimiento	Paso	El proceso total de bloqueo y persistencia toma una cota máxima de 2.5 segundos .
Frecuencia esperada	150 transacciones por jornada académica.	
Comentarios	El uso del bloqueo transaccional a nivel de fila evita condiciones de carrera concurrentes.	

RF-<id del caso>	Procesar Transacción de Devolución	
Versión	Versión 1.0 (Mayo 2026)	
Actores	Primario: Bibliotecario. Secundario: Alumno / Docente (Quien entrega el texto).	
Objetivos asociados	Registrar el retorno físico de los textos de la universidad y liberar la carga académica del alumno de forma inmediata.	
Requerimientos asociados	RF-04 (Registro Devolución), RF-05 (Cálculo de Multas y Penalizaciones).	
Descripción	El bibliotecario recibe el libro devuelto y escanea su código. El sistema verifica la fecha límite, calcula multas si corresponde, y libera el stock del libro.	
Pre-condición	El ejemplar ingresado debe estar registrado previamente en PostgreSQL con el estado actual de "Prestado".	
Secuencia Normal	Paso	Acción
	1	El bibliotecario recibe el ejemplar físico en el mesón de atención.
	2	Escanea el código de barras único adherido al texto con el lector láser.
	3	El sistema identifica automáticamente el ID del préstamo pendiente en la base de datos.

	4	El sistema evalúa si la fecha actual es menor o igual a la fecha límite estipulada.
	5	El sistema cambia el estado del libro de "Prestado" a "Disponible" en el inventario.
	6	El sistema elimina el cargo activo de la cuenta del alumno de forma definitiva.
	n	El sistema despliega un mensaje de éxito en pantalla y despacha un comprobante al correo del alumno.
Post-condición	El ejemplar físico queda liberado y visible de inmediato en el catálogo público en línea para nuevas búsquedas.	
Excepciones	Paso	Acción
	1	La fecha actual es posterior a la fecha máxima de entrega permitida.
	2	El sistema interrumpe el flujo normal y calcula los días de retraso.
	3	El sistema invoca a <i>MultaService</i> , añade la deuda financiera a la ficha del alumno y emite una alerta visual roja en la pantalla del bibliotecario.
Rendimiento	Paso	Cota de tiempo
		El tiempo de procesamiento lógico y persistencia en la base de datos debe ser menor a 1.5 segundos .
Frecuencia esperada	130 transacciones por día en jornadas ordinarias.	
Comentarios	Módulo crítico para normalizar con rapidez la disponibilidad real de los textos de alta demanda.	

RF-<id del caso>	Consultar Catálogo con Filtro Indexado
-------------------------------	--

Versión	Versión 1.0 (Mayo 2026)	
Actores	Primario: Alumno, Docente. Secundario: Ninguno.	
Objetivos asociados	Permitir la búsqueda remota de textos para disminuir las aglomeraciones y tiempos de espera en los mesones físicos.	
Requerimientos asociados	RF-06 (Catálogo en Línea), RNF-01 (Diseño Responsivo), RNF-04 (Tiempo de Consulta).	
Descripción	El estudiante ingresa términos de búsqueda en el portal web o móvil. El sistema realiza una consulta indexada en la base de datos y renderiza los libros con su ubicación exacta.	
Pre-condición	El portal web o aplicación móvil debe estar en línea y conectado con éxito al nodo de base de datos.	
Secuencia Normal	Paso	Acción
	1	El alumno abre la aplicación web o móvil de la biblioteca e ingresa a la pestaña del catálogo.
	2	Digita términos de interés (autor, título o área académica) en la barra de búsqueda.
	3	El sistema ejecuta una consulta indexada sobre la tabla de ejemplares en PostgreSQL.
	4	El sistema recupera los registros coincidentes de forma estructurada.
	5	El sistema renderiza las tarjetas de resultados mostrando portadas, títulos y stock actual.
	6	El usuario hace clic sobre un libro específico de la lista.
	n	El sistema revela el pasillo y estante exacto donde se encuentra guardado el material en la biblioteca.
Post-condición	El alumno conoce con precisión si el material está disponible y en qué estantería física buscarlo antes de ir al mesón.	
Excepciones	Paso	Acción
	1	No se encuentra ningún libro con los términos digitados en la barra.
	2	El sistema muestra por pantalla el mensaje informativo: "No se encontraron resultados para su búsqueda" y sugiere revisar la ortografía.

Rendimiento	Paso	Cota de tiempo
		La base de datos garantiza una cota de tiempo máxima de despliegue en pantalla menor a 3.0 segundos .
Frecuencia esperada	2.500 búsquedas por día estimadas en periodos de exámenes.	
Comentarios	Satisface directamente las pautas de accesibilidad y diseño responsivo móvil analizadas en el comité de calidad.	

3.1.6 Resultado Análisis de Calidad Diagramas Modelamiento

Se aplicó la Planilla de Análisis de Calidad sobre los diagramas UML estructurales y dinámicos (Clases, Componentes, Actividades y Despliegue) presentados en la Sección 1 de este informe, comprobando una conformidad del **100%** respecto a las especificaciones de notación de la OMG (Object Management Group). Se verificó con éxito la trazabilidad lógica: las funciones críticas del negocio descritas en los requerimientos funcionales mapean de forma exacta en métodos síncronos dentro de las clases de la vista lógica (`PrestamoService.java`) y en las ramas de decisión del diagrama de actividades, asegurando que no existan componentes huérfanos ni código redundante.

3.1.7 Resultado Análisis de Calidad Prototipado No funcional del Sistema

La inspección técnica heurística conducida sobre los wireframes de baja fidelidad demostró un alineamiento riguroso con los estándares de consistencia y prevención de errores. Las interfaces lógicas asignadas al mesón de atención optimizan el espacio de trabajo al situar el foco del cursor automáticamente en el campo de escaneo. Además, organizan las alertas críticas de morosidad escolar en el cuadrante superior con componentes visuales de alto contraste, logrando mitigar la fatiga visual de los operarios administrativos durante las jornadas de alta demanda.

3.1.8 Planillas entregables del Proyecto

A continuación, se detalla la planilla que identifica y define los módulos lógicos y los artefactos de software compilables que componen la solución arquitectónica de UNILIB, estableciendo los puntos de control para mitigar el impacto ante solicitudes de modificación:

Módulo Arquitectónico	Artefacto de Software / Código	Caso de Uso Asociado	Tipo de Artefacto	Descripción del Entregable

Módulo Arquitectónico	Artefacto de Software / Código	Caso de Uso Asociado	Tipo de Artefacto	Descripción del Entregable
unilib-frontend	CatalogoView.jsx	CU-03-001	Código Fuente (React)	Interfaz pública responsiva para la búsqueda y filtrado de libros por los alumnos.
unilib-backend-core	PrestamoService.java	CU-02-001 / CU-02-002	Código Fuente (Java)	Componente central que ejecuta las reglas de negocio de préstamos y bloqueos de stock.
unilib-backend-core	MultaService.java	CU-02-002	Código Fuente (Java)	Clase encargada de calcular de forma automática las penalizaciones por días de retraso.
unilib-db-scripts	V1__init_schema.sql	Todos los Casos de Uso	Script SQL (PostgreSQL)	Script DDL que define las tablas, llaves primarias e índices relacionales de la

Módulo Arquitectónico	Artefacto de Software / Código	Caso de Uso Asociado	Tipo de Artefacto	Descripción del Entregable
				biblioteca.

3.1.9 Matriz de Control de Cambios

D Cambio	Artefacto / Módulo Afectado	Razón o Motivo del Cambio	Impacto Arquitectónico Estimado	Estado
CC-01	MultaService.java	Modificar el valor de la multa diaria por atraso de \$1.000 a \$1.500 por políticas institucionales.	Bajo: Modificación simple en una variable de la lógica de negocio sin alterar el esquema de datos.	Aprobado
CC-02	CatalogoView.jsx	Incorporar un filtro de búsqueda avanzada para segmentar libros por año de edición y editorial.	Bajo: Cambio menor en la capa de presentación. Requiere actualizar el componente visual de React.	Aprobado
CC-03	Base de Datos (PostgreSQL)	Cambiar el motor transaccional SQL por una base de datos NoSQL (MongoDB) a mitad del desarrollo.	Crítico: Violación directa de la restricción de infraestructura y destrucción de la consistencia relacional.	Rechazado

3.1.10 Matriz EDT. Planilla Detallada Cálculo de Esfuerzo

Cód. EDT	Tarea Detallada del Proyecto UNILIB	Perfil / Rol del Integrante	Duración Estimada	Esfuerzo (Horas / Hombre)
1.1	Modelamiento de diagramas arquitectónicos y vistas 4+1	Diseñador de Software / UML	1 Semana	36 Horas
2.1	Diseño del modelo físico relacional y scripts de base de datos	Administrador de BD (DBA)	0.5 Semanas	18 Horas
3.1	Construcción de componentes, controllers REST y seguridad JWT	Desarrollador Backend (Java)	1.5 Semanas	54 Horas
4.1	Maquetación de vistas web responsivas y consumo de la API	Desarrollador Frontend (React)	1 Semana	42 Horas
5.1	Ejecución de pruebas integrales de QA, concurrencia y despliegue	Todo el Equipo (QA Testing)	0.5 Semanas	30 Horas
TOTAL	Ciclo de Vida de Diseño, Construcción y Aseguramiento	Equipo de 3 Alumnos	4 Semanas	180 Horas / Hombre

4 Prototipado de Software

Insertar Mockups y Wireframe de la interfaz de usuario del Sistema